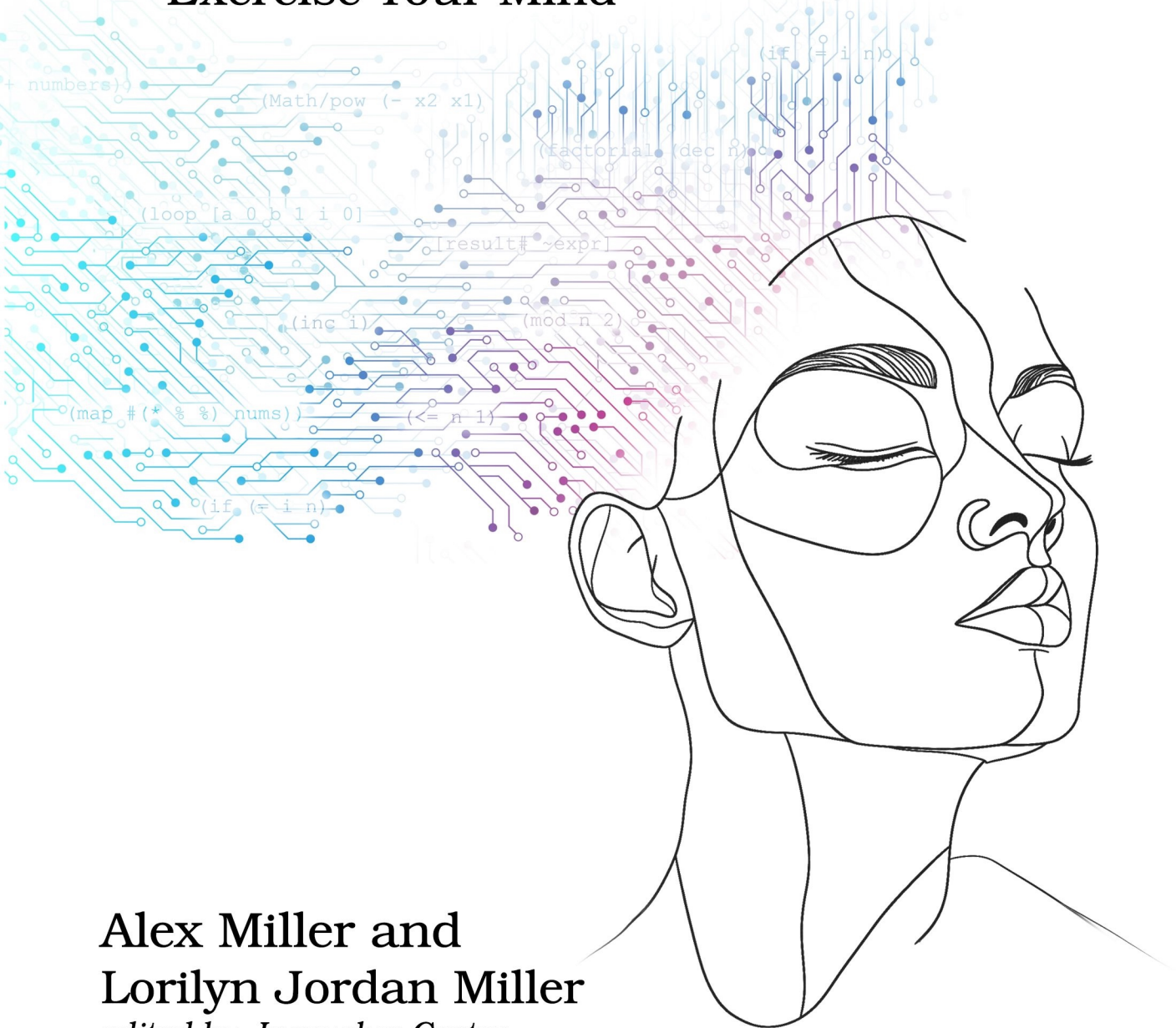


The
Pragmatic
Programmers

Clojure Brain Teasers

Exercise Your Mind



Alex Miller and
Lorilyn Jordan Miller
edited by Jacquelyn Carter

Early Praise for *Clojure Brain Teasers*

"Time and again, this book challenged my understanding of Clojure. Even after over 15 years with the language, it illuminated fascinating corners of the language I had never explored. Each teaser left me more curious than the last, eager to delve into Alex and Jordan's insightful explanations and uncover even more of Clojure's hidden secrets."

► **Eric Normand**

Author of *Grokking Simplicity*

"Much to my surprise, working my way through the teasers made me happy! I could literally feel how the exercises, along with the detailed explanations, helped the Clojure part of my mind to efficiently correct faulty assumptions and tune mental models."

► **Peter Strömberg**

Creator of Calva

"New/intermediate Clojure developers will benefit from this book, and it's an interesting (and unusual) presentation that is a lot of fun for all levels (including advanced Clojure developers)."

► **Sean Corfield**

Senior Software Architect

"Do you know Clojure as well as you think you do? Find out with this collection of delightful puzzles delivered with wisdom and mirth!"

► **Lee Read**

Enthusiast

"A playful adventure through the shadowy corners of one's knowledge, *Clojure Brain Teasers* introduced me to those treacherous gaps in my understanding that I had unwittingly paved over with the wrong assumptions and helped me puzzle through to the truth."

► **Epidiah Ravachol**

Owner, Dig a Thousand Holes Publishing

"This may look like a book of puzzles, but in fact, it is a fine whetstone upon which to hone your knowledge of Clojure. Even with a decade of Clojure experience, I was pleasantly surprised to fill in a few blind spots. Fun and functional!"

► **Rafal Dittwald**

VNTRHB

Clojure Brain Teasers

Exercise Your Mind

Alex Miller

Lorilyn Jordan Miller

The Pragmatic Bookshelf

Dallas, Texas



See our complete catalog of hands-on, practical,
and Pragmatic content for software developers:

<https://pragprog.com>

Sales, volume licensing, and support:

support@pragprog.com

Derivative works, AI training and testing,
international translations, and other rights:

rights@pragprog.com

The team that produced this book includes:

Publisher: Dave Thomas

COO: Janet Furlow

Executive Editor: Susannah Davidson

Development Editor: Jacquelyn Carter

Copy Editor: Karen Galle

Indexing: Potomac Indexing, LLC

Copyright © 2025 The Pragmatic Programmers, LLC.

All rights reserved. No part of this publication may be reproduced by any means, nor may any derivative works be made from this publication, nor may this content be used to train or test an artificial intelligence system, without the prior consent of the publisher.

When we are aware that a term used in this book is claimed as a trademark, the designation is printed with an initial capital letter or in all capitals.

The Pragmatic Starter Kit, The Pragmatic Programmer, Pragmatic Programming, Pragmatic Bookshelf, PragProg, and the linking g device are trademarks of The Pragmatic Programmers, LLC.

Every precaution was taken in the preparation of this book. However, the publisher assumes no responsibility for errors or omissions or for damages that may result from the use of information (including program listings) contained herein.

ISBN-13: 979-8-88865-129-2

Encoded using recycled binary digits.

Book version: P1.0—February 2025

Contents

<u>Acknowledgments</u>	vii
<u>Preface</u>	ix

Part I — Basics

<u>Puzzle 1.</u>	<u>All Things Being Equal</u>	3
<u>Puzzle 2.</u>	<u>You've Been Promoted</u>	7
<u>Puzzle 3.</u>	<u>Ba-NaN-a Peel</u>	11
<u>Puzzle 4.</u>	<u>Truthy or Dare</u>	15
<u>Puzzle 5.</u>	<u>Vacuums Suck</u>	19

Part II — Collections

<u>Puzzle 6.</u>	<u>Six of One, Half Dozen of Another</u>	25
<u>Puzzle 7.</u>	<u>When in Rome</u>	29
<u>Puzzle 8.</u>	<u>From Thin Air</u>	33
<u>Puzzle 9.</u>	<u>Needle in a Haystack</u>	37
<u>Puzzle 10.</u>	<u>Off the Record</u>	41
<u>Puzzle 11.</u>	<u>Out of Sorts</u>	45

Part III — Evaluation

<u>Puzzle 12.</u>	<u>Stay Focused, Reader</u>	51
<u>Puzzle 13.</u>	<u>Nil Comment</u>	55
<u>Puzzle 14.</u>	<u>Roll the Dice</u>	57
<u>Puzzle 15.</u>	<u>Paren-tal Advisory</u>	61
<u>Puzzle 16.</u>	<u>Quoth the Raven</u>	65
<u>Puzzle 17.</u>	<u>Loose Threads</u>	69

Part IV — Runtime and Library

Puzzle 18.	A Case of Mistaken Identity	75
Puzzle 19.	Boom Goes the Dynamite	79
Puzzle 20.	Banana Splits	85
Puzzle 21.	Hanging Around	89
Puzzle 22.	Build a Nest	93
Puzzle 23.	Partial Recall	97
Puzzle 24.	Abrac-OR-dabra	101
Puzzle 25.	Take the Hint	105
Bibliography		109
Index		111

Acknowledgments

Many thanks to Rafal Dittwald, Sean Corfield, Miran Tuhtan, Don Jennings, Epidiah Ravachol, Andrew Leverette, Eric Normand, and Lee Read for technical review and feedback on early drafts—the book is much better as a result of their careful reading!

Thanks to Rich Hickey for creating the beautiful and useful language Clojure and to everyone who has worked on the language since its inception.

Thanks to the Clojure community for inspiring many of the ideas for the teasers in this book—this book is both for you and from you!

Thanks to everyone at Pragmatic Bookshelf, especially our editor, Jacquelyn Carter.

Alex would like to thank his wife, Mary, for her patience while he spent yet more time at the computer. He would also like to thank Jordan for being a creative partner who always thinks about the how and why of teaching.

Jordan would like to thank the humans who inspire her. Alex Miller, who wields the power of Clojure to create life-changing opportunities. Robert Randolph for his leadership and for inspiring countless existential awakenings. Raf Dittwald for his dedication to helping people through pedagogy. Clojure Camp "Alpha Learners"—Wendy, Recia, Tommy, and Austin—and (begrudgingly!) James Tolton for offering me my first sip of clj Kool-Aid.

Preface

Clojure was designed to be a general-purpose functional language with a simple programming model for information systems (far simpler than the object-oriented models that dominate the industry). Many people have commented that aspects of Clojure, like immutable collections, functional programming, state management, concurrency, and open polymorphism, seem to fit together and support each other at a surprisingly deep level. Indeed, this is a consequence of the intense design effort that went into Clojure's initial creation and the attention the core team pays to design with every new feature.

However, some aspects of Clojure may defy expectations when coming from other languages or be unclear if the underlying design is not understood. The authors have spent many years assisting new developers in learning Clojure, and the same questions arise repeatedly. This book is designed to explore these common questions. Any new or intermediate Clojure developer will find familiar questions in these pages, and it is likely you will find some you have encountered in the past.

Additionally, Clojure is, by design, a *hosted language*, running on top of some other runtime, like the JVM, JavaScript, or the Microsoft CLR. Being hosted gives Clojure a huge advantage in library availability and integration with existing systems but also invites situations where Clojure and host semantics may differ.

About the Book

Each teaser starts with a simple program and asks you to guess the result. The teasers are designed to either produce a true or false or may, in some cases, throw an exception.

After you've turned the page, you'll find out what actually happens and get an explanation of why the program returns the answer it does, often with some discussion of related topics. These teasers are not "gotchas" and do not

exploit bugs in the language. Rather, these teasers reveal places where your prior experience or expectations have led you to a conclusion different than Clojure's design. But fear not; as you read this book, the underlying intent of the language will be revealed.

You will be able to use this knowledge in the future to better predict and leverage how your own Clojure programs work.

About the Authors

Alex Miller has been a member of the Clojure core team since 2013 and is the co-author of *Clojure Applied [VM15]* and *Programming Clojure, Third Edition [HB18]*. Alex is the creator or organizer of the Strange Loop, Lambda Jam, Clojure/west, and Clojure/conj conferences.

L. Jordan Miller is a Staff Software Engineer at Nubank, a Datomic Developer Advocate, and producer/host of the Lost In Lambduhhs podcast. Passionate about people, programming, and pedagogy, she's a conference speaker, host, organizer, and reviewer for events such as Heart of Clojure, Strange Loop, Re:clojure, and Clojure Conj.

About the Code

Each teaser starts with a small code snippet that can be run at the Clojure REPL (Read-Eval-Print Loop). The easiest way to run the Clojure REPL is to install the Clojure CLI¹ and then execute the command `clj` from your shell. If you're using a development environment, just open any REPL to type the teaser code.

In code examples, the REPL prompt is not printed, and instead, the expression is printed at the beginning of the line. Return values follow in a line starting `;;=>`. If output is printed (not returned), the line will start `;;`.

```
(+ 1 1)
;;=> 2

(println "Hello!")
;; Hello!
;;=> nil
```

In some cases, the output has been slightly modified to improve clarity, for example, by omitting return values that aren't important or simplifying exception messages.

1. https://clojure.org/guides/install_clojure

Each teaser's source code is also included as a standalone source file in the sample code for this book, available on the Pragmatic Bookshelf website.² There, you can download teaser code, participate in discussions, and report issues if needed.

About You

This book assumes some basic familiarity with Clojure and its features. If you are completely new to Clojure, we suggest that you pause each time you encounter something new and read about it in your favorite Clojure introductory text or online resource, like [Programming Clojure, Third Edition \[HB18\]](#) or [Getting Clojure \[Ols18\]](#). Because the topics are presented here example-first, you may find that they form a complementary path to learning.

You may want to read this while sitting by a REPL, trying each teaser before you turn the page. Or you may find it sufficient to just predict the answer in your brain REPL. Or you may just want to turn the page and read the answer, gauging your level of surprise. All of these are okay!

Have fun!

2. <https://pragprog.com/titles/mmclobrain>

Part I

Basics

*These teasers cover basic parts of the language
like equality, numerics, and logical truth.*

Puzzle 1

All Things Being Equal

```
book/src/all_things.clj
```

```
(= 1 1N 1.0 1.0M)
```

```
(== 1 1N 1.0 1.0M)
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(= 1 1N 1.0 1.0M)
;;=> false

(== 1 1N 1.0 1.0M)
;;=> true
```

Discussion

This teaser shows two different equality operators, and you can see from the results that they have different rules for equality. The single `=` operator is the normal operator you will use in most circumstances, and it works on values of any type to compare those values in a type-specific way.

Clojure has several numeric types—integers, big integers (indicated with `N` suffix), doubles, big decimals (indicated with `M` suffix), and ratios. The `=` operator will only compare two numbers of the same type. If you call `=` with numbers of different types, `=` will just simply report that as false. For example:

```
(= 2 2.0)
;;=> false
```

One exception to this is that big integers in the integer range **WILL** compare `=` to integers:

```
(= 1 1N)
;;=> true
```

In all other cases, `=` on values from different number types will compare false.

The `==` operator allows you to compare numeric values across classes by promoting one of the values to the larger or more expressive type before comparison. For example:

```
(== 2 2.0)
;;=> true
```

In this example, the first integer value `2` will be promoted to a double, and then the two double values will be compared as if the expression was:

```
(= (double 2) 2.0)
;;=> true
```

The `==` operator works only on numbers, so you can't use it on collections of numbers or anything else:

```
(== [1 2] [1.0 2.0])
```

```
;; Execution error (ClassCastException) at user/eval5 (REPL:1).  
;; class clojure.lang.PersistentVector cannot be cast to class java.lang.Number
```

Due to this constraint, == is relatively uncommon to see in Clojure code, but it's helpful in the case of cross-numeric comparison.

Further Reading

Clojure Reference - Numbers

https://clojure.org/reference/data_structures#Numbers

ClojureDocs - clojure.core/=

<https://clojuredocs.org/clojure.core/=>

ClojureDocs - clojure.core/==

<https://clojuredocs.org/clojure.core/==>

Puzzle 2

You've Been Promoted

[book/src/promotion.clj](#)

```
(def bignum 9223372036854775807)
(def biggernum 9223372036854775808)

(= biggernum (+ bignum 1))
(= biggernum (+ ' bignum 1))
(= biggernum (unchecked-add bignum 1))
```

Guess the Output



Try to guess what the output of these expressions are before moving to the next page.

The program will display the following output:

```
(= biggernum (+ bignum 1))
;; Execution error (ArithmeticException) at java.lang.Math/addExact
;; long overflow

(= biggernum (+ ' bignum 1))
;;=> true

(= biggernum (unchecked-add bignum 1))
;;=> false
```

Discussion

This teaser adds 1 in three different ways to a big number. Your first guess for any of these functions should be 9223372036854775808 (biggernum), but we see a variety of answers here.

The number bignum is not just any big number; it's a very special big number, $2^{63} - 1$, the largest possible primitive long value on the Java Virtual Machine. Integer numbers in Clojure are usually JVM long values, but may also be `clojure.lang.BigInt` values for integers too large to fit into a long. `BigInt` values are printed with a trailing 'N' suffix (like the second answer here):

```
bignum
;;=> 9223372036854775807

biggernum
;;=> 9223372036854775808N
```

Primitive numeric values fit in limited memory space (here, 8 bytes), so only so many numeric values can be represented. Primitive longs are in the range from -2^{63} to $2^{63} - 1$. Exceeding the bound of a numeric type is typically called *arithmetic overflow*, or on the other side, arithmetic underflow.

There are several possible approaches to bounds checking, and these correspond directly to the three examples in the teaser:

- Check for overflow and error (+)
- Check for overflow, and promote to a larger type (+')
- Don't check, and allow overflow (unchecked-add)

All of the Clojure mathematical operators have similarly named variants. The quote suffix can be read aloud as “promoting.”

The default arithmetic operators (such as +, -, *) all check for overflow and throw `ArithmeticException` when bounds are exceeded, so these operators are both

safe and (via low-level support) fast. Java provides methods for arithmetic operations with bounds checking in the `java.lang.Math` class. Clojure uses these methods internally, which is why you see it in the exception from the teaser.

The promoting arithmetic operators (such as `+`, `-`, `*`) all check for overflow, and when bounds are exceeded, promote the number to an arbitrary precision integer. Clojure integer numbers are always either JVM longs (8-byte, signed, primitive values) or arbitrary precision integers, which are not limited in size. You can imagine making such a data type by storing the number in a string with arbitrary length.

The promoting arithmetic operations are slower because they are not typically supported directly by the underlying CPU instruction set. Thus, you have a choice to make here between safety (via promotion) and speed. For performance, Clojure optimizes arithmetic operations of arbitrary-precision integers that are representable as longs.

Finally, the unchecked operations omit the check (slightly faster) but also may overflow. When overflow happens on a long, you will actually see it become the smallest possible number (this is also the default behavior for arithmetic operators in Java):

```
(unchecked-add bignum 1)
;;=> -9223372036854775808
```

Using unchecked operations in places that should never have negative values (any kind of count) is likely to break the assumptions of your code and lead to subtle bugs; thus, you should be extremely careful in where and how you choose to use unchecked operations. Another place where unchecked operations are commonly allowed is in hash or cryptographic algorithms, where the overflow is expected and necessary.

Further Reading

Integer Overflow - Wikipedia

https://en.wikipedia.org/wiki/Integer_overflow

Clojure Reference - Java Primitives

https://clojure.org/reference/java_interop#primitives

Clojure Reference - Reader

<https://clojure.org/reference/reader>

ClojureDocs - `clojure.core/+`

<https://clojuredocs.org/clojure.core/+>

ClojureDocs - *clojure.core/+*'

<https://clojuredocs.org/clojure.core/+>

ClojureDocs - *clojure.core/unchecked-add*

<https://clojuredocs.org/clojure.core/unchecked-add>

Puzzle 3

Ba-NaN-a Peel

```
book/src/banana.clj
```

```
(= ##NaN ##NaN)
```

```
(== ##NaN ##NaN)
```

Guess the Output



Try to guess what the output of this expression is before moving to the next page.

The program will display the following output:

```
(= ##NaN ##NaN)
;;=> false

(== ##NaN ##NaN)
;;=> false
```

Discussion

Clojure has a few special *symbolic values*, which all start with `##` — `##NaN`, `##Inf`, and `##-Inf`, representing the Java static fields `Double/NaN`, `Double/POSITIVE_INFINITY`, and `Double/NEGATIVE_INFINITY` respectively.

NaN is a special abbreviation from the IEEE 754 floating point standard meaning “Not a Number.” What? How can a number be “not a number”? There are a few cases in floating point math that result in a value that cannot be represented, and thus they are `##NaN`:

```
(require '[clojure.math :as math])
;;=> nil
(math/sqrt -1.0)
;;=> ##NaN
(/ 0.0 0.0)
;;=> ##NaN
```

With that background, let’s return our attention to the teaser. `##NaN` does not represent any specific number, so comparing two `##NaN` values is like comparing two unknown values, and this must be false.

While you are likely not comparing two `##NaN` values directly, you might end up comparing two collections that contain a `##NaN` value deep in a data structure as a result of a computation on doubles (dividing by 0.0, for example). When you are working on code that transforms doubles, keep this in mind.

The other less common but more tricky case is that hashed data structures (maps and data sets) find keys based on the hash value and then compare based on equals. You can use `##NaNs` as keys to these data structures, but they will never be found or retrieved, which can lead to some very unexpected behavior.

```
(def m {##NaN 100})
;;=> #'user/m
(count m)
;;=> 1
(get m ##NaN) ;; expecting 100
;;=> nil
```

So, don't slip on that ba-NaN-a peel! Watch out for ##NaNs.

Further Reading

IEEE 754-1985 Floating Point Specification - NaN (Not a Number)

https://en.wikipedia.org/wiki/IEEE_754-1985#NaN

Clojure Guide - Equality

<https://clojure.org/guides/equality>

Clojure Reference - Reader

<https://clojure.org/reference/reader>

Java Platform API - java.lang.Double

<https://docs.oracle.com/javase/8/docs/api/java/lang/Double.html>

Puzzle 4

Truthy or Dare

[book/src/truthy1.clj](#)

```
(true? true)
(true? :sky-is-blue)
(false? false)
(false? nil)
(false? `())
(false? 0)
```

Guess the Output



Try to guess what the output of these expressions are before moving to the next page.

The program will display the following output:

```
(true? true)
;;=> true
(true? :sky-is-blue)
;;=> false
(false? false)
;;=> true
(false? nil)
;;=> false
(false? `())
;;=> false
(false? 0)
;;=> false
```

Discussion

Clojure has a concept of “logical true” and “logical false”; many people refer to these colloquially as “truthiness” and “falsiness.” This means that everything except the values `false` and `nil` implicitly resolve to `true` in a conditional expression or when coerced into a Boolean using `(boolean x)`.

Conversely, the predicate functions `(true? x)` and `(false? x)` check if the values of `x` are specifically the *Boolean values* `true` or `false`, which is distinct from the concepts of “logical truth” and “logical false.”

You can use `and`, `or`, and `not` to combine logical values. One important and common use is to check whether a value is `nil` by implicitly using it as a truth value.

Understanding this can improve your code style. For example, to check if the value `moms-birthday` exists, and if so, print “Happy Birthday Mom!!”, we have a few options. Each of these conditional functions has the same output, but one of them is typically preferred over the others.

```
(def moms-birthday "April 20, 1969")
(when-not (nil? moms-birthday)
  (println "Happy Birthday Mom!!"))
(when (some? moms-birthday)
  (println "Happy Birthday Mom!!"))
(when moms-birthday
  (println "Happy Birthday Mom!!"))
```

In the last example, `moms-birthday` is used as a “logical truth.” In contrast to the other more cumbersome expressions, it’s the most concise and readable. Remember this when reaching for function conditional operations such as `if`, `when`, `cond`, and so on.

Further Reading

ClojureDocs - clojure.core/true?

https://clojuredocs.org/clojure.core/true_q

“Clojure Truthy and Falsey” by Jay Fields

<http://blog.jayfields.com/2011/02/clojure-truthy-and-falsey.html>

Puzzle 5

Vacuums Suck

```
book/src/vacuums_suck.clj
```

```
(and)  
(every? odd? [])  
(or)
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(and)
;;=> true

(every? odd? [])
;;=> true

(or)
;;=> nil
```

Discussion

Both the `and` and `every?` functions return `true` due to a concept from logic known as “vacuous truth.” This term describes a situation where a logical statement is true because no examples meet the condition to evaluate it. If there are no examples, there are no false examples; therefore, the statement is true.

Imagine you have a robot programmed to vacuum all the carpets in a house. If your house has exclusively hardwood floors, the robot will complete its task and report that it has vacuumed all the carpets. The statement “the robot vacuumed all the carpets” is technically true, but only because there were no carpets to begin with. This is an example of vacuous truth, where the statement is true in a trivial sense.

In the code, both `and` and `every?` return `true` because there are no arguments supplied. When these functions search for arguments to compare and come up empty, they are similar to the robot searching for carpets to vacuum. Since there is nothing to operate on, in both situations, the operator defaults to `true`.

On the other hand, the `or` function behaves differently. When `or` is called with no arguments, it returns `nil` instead of `true`. This is because `or` finds the first truthy value among its arguments or returns `nil` if none are found. With no arguments, there is no value to return, so `or` defaults to `nil`. Understanding this behavior helps in writing more predictable conditional logic in Clojure.

You might wonder why `or` defaults to `nil` instead of `false`. Both `and` and `or` return actual logically true values from their expressions instead of `true` or `false`, allowing `and` and `or` to be used as part of a value initializing expression. `and` returns the last logically true value (if all are logically true). `or` returns the first logically true value if any are true (and the remaining expressions are not evaluated):

```
;; returns last logically true value  
(and 1 2)  
;;=> 2  
  
;; returns first logically true value  
(or nil false 1 2)  
;;=> 1
```

For (and), there is no value to return, and only true makes sense as a default truthy value. With (or), either nil or false are options, but nil is more useful as a base value when combining 0 or more expressions into a value.

Other programming languages, such as JavaScript and Python, also have examples of “vacuous truth” logical operations.

Further Reading

Vacuous Truth - Wikipedia

https://en.wikipedia.org/wiki/Vacuous_truth

Part II

Collections

Clojure is a data-oriented language, built on immutable collections (composite values). These teasers check your understanding of the Clojure collection types.

Puzzle 6

Six of One, Half Dozen of Another

[book/src/six_of_one.clj](#)

```
(= [0 1 2 3 4 5]  
   '(0 1 2 3 4 5)  
   (range 6))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(= [0 1 2 3 4 5]
   '(0 1 2 3 4 5)
   (range 6))
;=> true
```

Discussion

In Clojure, all collections are compared based on their contents. The example shows three different kinds of sequential collections—a vector, a list, and a sequence—each with the same elements. Because these are the same elements in the same order, they compare as equal.

There are, fundamentally, three kinds of Clojure collections for the purposes of equality: sequential, set, and map. Any collection within one of these partitions will compare as equal to another collection within the same partition if it contains the same elements.

Sequential collections compare the elements of the collection in sequential order. Sets check that one set contains exactly the elements in the other set (order is not relevant). Maps check that one map contains exactly the same keys mapped to the same values in the other map (order is not relevant).

Comparing collections from different equality partitions will always compare false:

```
(= [0 1 2 3 4 5] #{0 1 2 3 4 5})
;=> false
```

Here, a vector (sequential partition) and set (set partition) contain the same elements but will never compare equal, even for empty collections.

These rules work recursively, so they can be applied to any depth of any nested collection as well.

Note that these rules also apply when comparing Clojure collections to Java collections; for example, a Java `ArrayList` will compare equal, by contents, to a Clojure vector. An important exception is Java arrays, which are not collections and are compared by identity rather than value. You can use `seq` to convert an array to a sequence and compare to a Clojure or Java collection by value.

Further Reading

ClojureDocs - [clojure.core/](https://clojuredocs.org/clojure.core/)=

<https://clojuredocs.org/clojure.core/>=

Clojure Guide - Equality

<https://clojure.org/guides/equality>

Puzzle 7

When in Rome

[book/src/when_in_rome.clj](#)

```
(= (conj '(:colosseum :vatican) :pantheon :trevi-fountain)
   (conj [:colosseum :vatican] :pantheon :trevi-fountain))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(= (conj '(:colosseum :vatican) :pantheon :trevi-fountain)
   (conj [:colosseum :vatican] :pantheon :trevi-fountain))
;;=> false
```

Discussion

As we plan our itinerary of sights in Rome, we decide to add the Pantheon and the Trevi Fountain to our stops. But, as we can see, it matters a lot whether we are using lists or vectors to store those sights:

```
(conj '(:colosseum :vatican) :pantheon :trevi-fountain)
;;=> (:trevi-fountain :pantheon :colosseum :vatican)
(conj [:colosseum :vatican] :pantheon :trevi-fountain)
;;=> [:colosseum :vatican :pantheon :trevi-fountain]
```

The `conj` function (short for “conjoin”) is used to add one or more elements to a collection. Each collection type inserts elements in the way that is most efficient for the collection, not necessarily at the beginning or end. In other words, `conj` does not mean “append” or “prepend.”

The most common point of confusion for new Clojure users is why lists (or sequences) insert elements at the beginning and vectors insert elements at the end. This all goes back to their implementation and where it is efficient to add elements.

Clojure lists or sequences are conceptually implemented as linked lists, a data structure that can be thought of as a chain where each link in the chain consists of a value and a reference to the next link. To traverse the data structure, we start at the head and walk the chain through the references to get to the end.

Inserting a new element could be done at the end, but only if you start at the head of the chain, walk through every link, and then construct a new link with the element at the end. The time to do this is proportional to the length of the list (1000 units of time for a list with 1000 elements). However, inserting at the beginning always takes a constant amount of time—you make a new link that simply refers to the head of the existing chain. `conj`ing multiple elements will do this repeatedly, so the last thing we added (`:trevi-fountain`) is the first thing in the resulting list.

Vectors, however, are stored as a tree (technically, a “hash array-mapped trie,” a data structure invented by Phil Bagwell). We won’t go into those details

here, but the tree is arranged in the order of the vector elements. Adding a new value can easily be done at the end by walking the right-most nodes of the tree and appending a new value to the final hash node leaf.

The time it takes to do the tree traversal is proportional to the height of the tree, and the height is the log, base 32, of the number of elements in the vector. Because this data structure uses relatively wide internal nodes, the tree is very shallow, and the height of the tree is usually only a few levels deep, usually no more than 3. Because the height grows so slowly, we sometimes call the traversal time “effectively constant.”

Inserting an element at the beginning of the vector could also be relatively fast, but it would require some additional overhead due to the implementation. More importantly, it makes more semantic sense to add elements to the end of the indexed vector than the beginning (or all of the indices would change). For these reasons, vectors insert elements at the end.

Retaining Order While Transforming

Because lists and vectors have different behavior for `conj`, it is important to keep track of which one you are using at any given time in your code. Generally, because vectors are easier to type in (no quoting necessary) and `append` at the end, data is most commonly built up using vectors and then transformed using sequence functions like `map` and `filter`, which return lazy sequences.

For the specific cases of `map` and `filter`, there are also `mapv` and `filterv` variants of `map` and `filter` which return vectors:

```
;; with map, returns sequence
(map #(* % 3) (range 5))
;;=> (0 3 6 9 12)

;; with mapv, returns vector
(mapv #(* % 3) (range 5))
;;=> [0 3 6 9 12]
```

A more general solution is to use *transducers* and `into`. Most sequence functions provide an additional arity that omits the collection and instead returns a transducer, a function that embodies the function algorithm but decouples from the input and output sources. Transducers can be composed using `comp` into a chain of transformations applied to a source of values, like a collection.

The `into` function takes a target collection, an optional transducer, and a source collection. You can use `into` with a `map` transducer to collect the results into a vector like this:

```
(into [] (map #(* % 3)) (range 5))
;;=> [0 3 6 9 12]
```

Note that `map` here takes only the mapping function. `into` is responsible for applying the resulting transducer function to the source and storing the result into the output vector. This is more complicated than `mapv` but also more flexible (when composing multiple transducers) and potentially faster in better avoiding intermediate collections.

Sets

It's also interesting to consider adding elements to a set:

```
(conj #{:colosseum :vatican} :pantheon :trevi-fountain)
;;=> #{:vatican :trevi-fountain :colosseum :pantheon}
```

The important thing to remember about sets is that they guarantee no particular order. The actual order they are printed in ultimately derives from the hash codes of the elements, which have no obvious relationship to the values (and that order may even differ between Clojure versions). The other interesting aspect of sets is that you cannot add the same element more than once. Mathematically, a set is a set of values (duplicates are not possible):

```
(conj #{1 2 3} 4 4 4)
;;=> #{1 4 3 2}
```

You can try to add the same value multiple times, but it is only added for the first one and makes no modification after that.

Further Reading

ClojureDocs - [clojure.core/conj](https://clojure.org/core/conj)

<https://clojuredocs.org/clojure.core/conj>

Clojure Reference - [Data Structures](https://clojure.org/reference/data_structures)

https://clojure.org/reference/data_structures

“Ideal Hash Trees” by Phil Bagwell

<https://infoscience.epfl.ch/record/64398/files/idealhashtrees.pdf>

Clojure Reference - [Transducers](https://clojure.org/reference/transducers)

<https://clojure.org/reference/transducers>

Puzzle 8

From Thin Air

[book/src/from_thin_air.clj](#)

```
(nil? (assoc nil :ex :nihilo))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(nil? (assoc nil :ex :nihilo))  
;;=> false
```

Discussion

In this teaser, you won't get `nil` back. Instead, a map will spring into existence "from thin air," or as in the Latin phrase, "ex nihilo":

```
{:ex :nihilo}
```

In Clojure, the core collection functions like `conj` and `assoc` are *polymorphic*, meaning that these functions accept collections of many types and do the operation appropriate to that collection type.

In addition to working with different kinds of collections, most Clojure collection functions are also defined for `nil` inputs and perform some useful operation. For functions that create data structures, a new data structure will be created: a map for `assoc` and a vector for `conj`. In other operations that retrieve or look up values, `nil` acts as an empty version of a collection and returns a useful result:

```
(count nil)  
;;=> 0  
  
(nth nil 0)  
;;=> nil  
  
(contains? nil :some-key)  
;;=> false  
  
(get nil :some-key)  
;;=> nil  
  
(first nil)  
;;=> nil  
  
(rest nil)  
;;=> ()
```

This polymorphic `nil` behavior is one of the little details that make Clojure data manipulation code so nice to work with and removes a whole class of `nil?` checks that would be needed otherwise.

Java Interop

One place where `nil` is NOT handled automatically is in Java interop, including working with Java strings via the `clojure.string` namespace functions. When invoking a Java function and passing `nil` for the string value, you will get the same behavior you would see when making that invocation in Java (usually a `NullPointerException`):

```
(require '[clojure.string :as str])  
(str/capitalize nil)  
;; Execution error (NullPointerException) at user/eval10 (REPL:1).
```

This is done both to retain the semantics of Java and also to avoid introducing any additional wrapper code in the execution path for interop calls. However, it does require a bit more care if you are using the Java collections directly or invoking a series of Java method calls.

Further Reading

Clojure Reference - Data Structures

https://clojure.org/reference/data_structures

Puzzle 9

Needle in a Haystack

[book/src/needle_in_a_haystack.clj](#)

```
(def haystack  
  (shuffle (conj (range 100) :needle)))  
(contains? haystack :needle)
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(def haystack
  (shuffle (conj (range 100) :needle)))
(contains? haystack :needle)
;;=> false
```

Discussion

It is hard to find a needle in a haystack, but it certainly seems like we should have found it with this code! This teaser creates a randomized vector of the numbers 0-99 and the keyword `:needle`. The confusion here lies in the `contains?` function.

When a function in `clojure.core` doesn't do what you're expecting, your first stop should always be to read the docstring, which may explain more:

```
(doc contains?)
;;-----
;;clojure.core/contains?
;;([coll key])
;; Returns true if key is present in the given collection; otherwise
;; returns false. Note that for numerically indexed collections like
;; vectors and Java arrays, this tests if the numeric key is within the
;; range of indexes. 'contains?' operates constant or logarithmic time;
;; it will not perform a linear search for a value. See also 'some'.
```

The answers are all here—`contains?` is about searching for keys, not values, and for indexed collections (like the vector we have), the keys are the indices of the vector. So, this function expects a key, but we're giving it a value. It's never going to work!

This leads us to the next question—how do we find whether a collection contains a value?

Indexed Data Structures

Clojure has several indirect solutions to this but no exact answer in the common library. In general, if you need to check for the existence of a value in a data structure, you should prefer to use a collection with that value as a key, like a map or a set.

In this case, we could instead build a set (which cannot be shuffled because sets have no order):

```
(def haystack
  (set (conj (range 100) :needle)))

(contains? haystack :needle)
;;=> true
```

Sets act as indexed collections of key to key. This code turns the values into keys of a set, which can be queried with `contains?`. Importantly, this is a fast lookup vs. the linear scan that would have been required in the original code. This is a case where Clojure nudges you into a better solution.

If your data was more complex, you might instead create a map keyed by the attribute you care about.

Linear Search

Sometimes, it is inconvenient or slower to convert your data into a data structure that is keyed on the lookup value, and you really must find a value by walking through a data structure to get it. For example, if your data is an infinite sequence, it is not possible to move it into a data structure, but it is easy to inspect values until you find a match (and then stop).

In this case, there are two main paths in common use. The first alternative is to filter a collection (as a sequence) to matching values and then pull only the first value:

```
(first (filter #{:needle} haystack))
;;=> :needle
```

Because `filter` is lazy, not all data needs to be processed. This function will return when it finds the first `:needle` value. However, because `filter` is a chunked sequence, the work is amortized in batches, and `filter` may process a whole “chunk” (usually up to 32 elements) at a time before the first is evaluated. This is important if you are streaming data from a file or external input source and want to avoid reading any more than necessary.

Recall also that `filter` works by finding values that match a predicate. Here, we are using a set as the predicate to enumerate the set of values to match. Sets are functions of their members—if the set contains the value, it returns that value.

Another more direct approach is to use the `some` function, which returns the first logically true value in a sequence:

```
(some #{:needle} haystack)
;;=> :needle
```

This is similar in function to the filter approach, but some does not use a chunked sequence, so it may be better if reading an expensive or external resource.

Predicates

Both filter and some apply a predicate to each value in turn. They differ in that some returns the (pred x) value, whereas first/filter returns x. In the previous example, we are using a set as the function, where (= (pred x) x) when the set contains the element. In other cases, that may not be the case:

```
;; returns the first value matching the predicate
(first (filter odd? (range 10)))
;;=> 1

;; returns the first logically true result of the predicate
(some odd? (range 10))
;;=> true
```

some is not generally a tool for linear search; it is the logical existential quantifier, checking whether “there exists” an element matching the predicate and also happens to return that predicate’s result. The prior set predicate just happens to be a case where the original element is returned (if it is contained in the set).

Further Reading

Clojure Reference - Data Structures

https://clojure.org/reference/data_structures

ClojureDocs - clojure.core/contains?

https://clojuredocs.org/clojure.core/contains_q

ClojureDocs - clojure.core/filter

<https://clojuredocs.org/clojure.core/filter>

ClojureDocs - clojure.core/some

<https://clojuredocs.org/clojure.core/some>

Off the Record

```
book/src/off_record.clj
```

```
(defrecord Album [name artist])

(def news-of-the-world
  (->Album "News of the World" "Queen"))

(instance? Album (dissoc news-of-the-world :artist))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(defrecord Album [name artist])
(def news-of-the-world
  (->Album "News of the World" "Queen"))
(instance? Album (dissoc news-of-the-world :artist))
;;=> false
```

Discussion

In this teaser, we create a record representing an album and an album instance, specifically the excellent Queen album, “News of the World.” If removing the `:artist` field returns something other than an `Album`, what is it?

```
(dissoc news-of-the-world :artist)
;;=> {:name "News of the World"}
```

From the example, we see that the result is just a plain map. Records in Clojure are maps but with two additional features: a concrete type (the name of the record) and a base set of required fields (treated as keyword keys). While records may have additional fields, they are guaranteed to have the specified fields, and their lookup is optimized for performance.

The `dissoc` function removes a mapping from a map based on the key. Removing a required key from a record would result in a record without the required field, so record instances devolve to a plain map. The other similar operation in Clojure with this behavior is `empty`, which creates a new instance of a map while removing all the mappings.

An alternate way to perform this operation while retaining the type is to leave the mapping but `assoc` a `nil` value to it.

```
(assoc news-of-the-world :artist nil)
;;=> #user.Album{:name "News of the World", :artist nil}
```

But really, just don’t take Queen off the record. They rock!

Further Reading

Clojure Reference - Data Structures

https://clojure.org/reference/data_structures

ClojureDocs - clojure.core/defrecord

<https://clojuredocs.org/clojure.core/defrecord>

ClojureDocs - clojure.core/dissoc

<https://clojuredocs.org/clojure.core/dissoc>

Out of Sorts

[book/src/out_of_sorts.clj](#)

```
(contains? (sorted-set 1 2 3) :hello)
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(contains? (sorted-set 1 2 3) :hello)
;; Execution error (ClassCastException) at user/eval1 (REPL:1).
;; java.lang.Long cannot be cast to clojure.lang.Keyword
```

Discussion

In this teaser, we create a sorted set of numbers and then try to find the keyword `:hello`. Because the comparator function for this set can't compare numbers and keywords, the collection's comparator throws an exception.

Clojure provides sorted collections that sort by key (for maps) or element (for sets). These are created using the `sorted-map` or `sorted-set` constructor functions, which optionally take a *comparator function*.

Comparator Functions

The comparator function is used to compare values and order the values in the collection. Clojure-sorted collections can use two comparator function styles: a comparator that returns an integer (Java style) or a comparator that returns a Boolean.

A simple Java-style comparator function for numbers might look like this:

```
(defn num-comparator
  [i1 i2]
  (- i1 i2))

(num-comparator 1 1)
;;=> 0

(num-comparator 1 10)
;;=> -9

(num-comparator 10 1)
;;=> 9

(sorted-set-by num-comparator 2 1 5 3 4)
;;=> #{1 2 3 4 5}
```

The Java-style comparator returns an integer whose sign describes the ordering of the two values: a negative sign indicates the first value is less than the second, and a positive sign indicates the reverse. 0 indicates the two values are equal. All sort algorithms can use a comparator like this to compare pairs of values while the values are sorted. Note that the magnitude of the positive or negative number is not used, just the sign.

The Boolean comparator is a simpler form—it is expected to return true if the first value is less than the second value and false otherwise. This makes it easier to write, but it may require multiple invocations to differentiate all three cases.

```
;; Easier!
(defn num-comparator-bool
  [i1 i2]
  (< i1 i2))

(sorted-set-by num-comparator-bool 2 1 5 3 4)
;;=> #{1 2 3 4 5}
```

A Java-style comparator can easily be created from a Boolean-style comparator, and this is effectively what Clojure does:

```
;; Converting Boolean-style comparator to Java-style comparator
(defn to-java-comparator
  [bool-comparator]
  (fn [v1 v2]
    (if (bool-comparator v1 v2)
      -1
      (if (bool-comparator v2 v1)
        1
        0))))

(sorted-set-by (to-java-comparator num-comparator-bool) 2 1 5 3 4)
;;=> #{1 2 3 4 5}
```

Boolean comparators are often easier to write but require extra invocations during sorting to distinguish the = and > cases.

Default Comparator

The default comparator function for Clojure sorted collections is `clojure.core/compare`. The `compare` function compares two values, which can be numbers, Clojure types, or any Java type that supports the `Comparable` interface, such as `String`, or `Instant`. The `nil` value always sorts first (less-than) when compared to a non-`nil` value.

While the `compare` function can be used with values of many different types (such as numbers, keywords, or strings), it is important that the values be of the same type. The `compare` function does not define an ordering between two values of different types (a number and a string). Thus, `compare` is a *partial order* (orders values of particular types), not a *total order* (orders ALL values).

See the example on the next page.

```
(compare :a :b) ;; two keywords (same type)
;;=> -1
(compare 1 2)   ;; two numbers (same type)
;;=> -1
(compare :a 1)  ;; keyword and number (different types)
;; Execution error (ClassCastException) at user/eval5 (REPL:1).
;; class java.lang.Long cannot be cast to class clojure.lang.Keyword
```

Thus, sorted collections using the default comparator function should only include keys of the same type. In this teaser, the comparator function will be invoked when comparing the lookup value (`:hello`) and values in the set (numbers). The default comparator does not define an ordering between keywords and numbers (which one is "higher"), so an exception is thrown from the comparator.

In practice, this does not happen often, but it can be surprising when it does. Receiving a key of a different type is often the result of a bug, but if the value comes from an external source, you may need to check its type before performing a lookup on the sorted collection.

Custom Comparators

An alternative approach is to supply your own comparator function that defines a total order (or at least an ordering between all of the key types you are concerned about). For example, you could create a comparator function that converts all values to strings and then compares those strings with the `compare` function:

```
(defn total-compare [x y] (compare (str x) (str y)))
(sorted-set-by total-compare :hello 2 "x" 1 10 false)
;;=> #{1 10 2 :hello false "x"}
```

This example converts each value to a string and then compares those strings. Note that this is sorting characters, not numbers, so `"1" < "10" < "2"`.

We can now revisit the original teaser and see how a comparator with total order now handles this case:

```
(contains? (sorted-set-by total-compare 1 2 3) :hello)
;;=> false
```

Depending on your needs, you can construct your own comparators that sort as needed.

Further Reading

Clojure Guide - Comparators

<https://clojure.org/guides/comparators>

Part III

Evaluation

Understanding how Clojure code is executed relies on understanding read and eval. The reader parses characters (text) and creates Clojure data representing the code. The evaluator takes Clojure code (as data) and evaluates the form, producing a result value. These teasers highlight various aspects of read and eval.

Stay Focused, Reader

`book/src/stay_focused.clj`

```
(def my-msgs
  {:emails [[:from "boss"] [:from "mom"]
             #_ #_ [:from "Nigerian Prince"] [:from "LinkedIn"]]
   :discord-msgs {"Clojure Camp" 6
                  #_ #_ "Heart of Clojure" 3
                  "DungeonMasters" 20}
   :voicemails ["Your voicemail box is full."]})

(defn unread [msgs]
  (let [{:keys [emails discord-msgs voicemails]} msgs]
    (+ (count emails)
       (reduce + (vals discord-msgs))
       (count voicemails))))

(= 34 (unread my-msgs))
```

Guess the Output



Try to guess what the output of these expressions are before moving to the next page.

The program will display the following output:

```
(def my-msgs
  {:emails [[:from "boss"] [:from "mom"]
             #_ #_ [:from "Nigerian Prince"] [:from "LinkedIn"]]
   :discord-msgs {"Clojure Camp" 6
                  #_ #_ "Heart of Clojure" 3
                  "DungeonMasters" 20}
   :voicemails ["Your voicemail box is full."]})

(defn unread [msgs]
  (let [{:keys [emails discord-msgs voicemails]} msgs]
    (+ (count emails)
       (reduce + (vals discord-msgs))
       (count voicemails))))

(= 34 (unread my-msgs))
;;=> false
```

Discussion

What do we actually receive?

```
(unread my-msgs)
;;=> 29
```

In this teaser, we define a function `unread` that simulates computing an unread message count from a set of pending notifications, such as emails, Discord chat messages, and voicemails. The `my-msgs` data holds specific data structures representing each notification type. Some notifications are discarded using the discard reader `#_`. The discard reader is a Clojure feature that allows you to skip specific code expressions without sending them to the reader for evaluation, making it particularly useful for debugging and experimentation.

Using the discard reader `#_` in Clojure is like applying a filter to ignore the next expression. It's particularly useful within `let` bindings of Clojure data structures when you want to disable parts of the data without removing them entirely. This is especially handy for debugging and experimenting with code changes, allowing you to test different parts of your code without the hassle of removing and re-adding expressions. When you stack discard readers, you can skip over multiple expressions in sequence, making it even easier to manage which parts of your code get executed.

Here's a breakdown of what happens in `my-msgs`:

1. **Emails:** Includes a vector of people who have sent you emails. Two consecutive `#_` discard readers are used, which means the next two expressions are discarded. Thus, `[:from "Nigerian Prince"]` and `[:from "LinkedIn"]` are both ignored. The resulting list contains two emails from “boss” and “mom.”
2. **Discord Messages:** The list of Discord messages contains three entries, one for each Discord server. The second entry, “Heart of Clojure” 3, is preceded by two discard symbols `#_`, so both the key and value are ignored. This is a common idiom in maps and let bindings where values are paired.
3. **Voicemails:** Binds voicemails to one voicemail.

When `unread` computes the unread messages, it sees two (not four) emails, 26 (not 29) Discord messages, and one voicemail, for a total of 29.

Imagine the Clojure reader as a notification filter, automatically skipping over the alerts that you don’t need to focus on the essential ones. This way, you can keep your attention on what truly matters without being distracted by unnecessary details.

Further Reading

Clojure Reference - Reader

<https://clojure.org/reference/reader>

“Don’t Say Homoiconic” by Klaas van Schelven

<https://www.expressionsofchange.org/dont-say-homoiconic/>

Nil Comment

[book/src/nilcomment.clj](#)

```
(map inc [1 2 3 (comment 4) 5])
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(map inc [1 2 3 (comment 4) 5])
;; Error printing return value (NullPointerException)
;; at clojure.lang.Numbers/ops (Numbers.java:1095).
;; Cannot invoke "Object.getClass()" because "x" is null
```

Discussion

This is why you shouldn't throw Clojure comment blocks around all willy-nilly. Distinct from other kinds of commenting forms, the comment macro evaluates and returns a value, `nil`. When applying the function to each element of the vector, an exception is thrown when `inc` attempts to increment the result of `(comment 4)`, which is `nil`.

Use comment blocks only in places where the evaluation to `nil` does not introduce a new argument to a surrounding expression, such as at the top level of your namespace. Clojure developers sometimes leave a comment block containing meaningful code samples below a function or namespace as a hint of how to use particularly complex or domain-specific code.

```
(defn my-function [x] x)

(comment
  ;; test my-function
  (my-function 100))
```

These “Rich Comment Blocks” can serve as helpful breadcrumbs, allowing others to gain insight as to how a function is used. Comment blocks can also be leveraged as working space during REPL-driven development, but these blocks typically don't make it into production code. Many Clojure development environments are aware of top-level comment blocks and provide special formatting or support there.

Be careful to remember that if you are debugging source code and trying to omit an expression, the discard reader symbol `#_` is a much better choice.

Further Reading

ClojureDocs - clojure.core/comment

<https://clojuredocs.org/clojure.core/comment>

“Running With Scissors: Live Coding With Data” by Stuart Halloway (Strange Loop 2018)

<https://www.youtube.com/watch?v=Qx0-pViyIDU&t=1229>

Roll the Dice

[book/src/roll_dice.clj](#)

```
(= #{(rand-int 6) (rand-int 6)}  
   #{(rand-int 6) (rand-int 6)})
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(= #{(rand-int 6) (rand-int 6)}
  #{(rand-int 6) (rand-int 6)})
;; Syntax error reading source at (REPL:10:32).
;; Duplicate key: (rand-int 6)
;; Syntax error reading source at (REPL:10:32).
;; Duplicate key: (rand-int 6)
```

Discussion

In this teaser, we use `(rand-int 6)` to simulate the roll of a single die and a set `#{(rand-int 6) (rand-int 6)}` to represent rolling two dice. Are two random dice rolls the same? You might think this should randomly return either `false` or, if we're very lucky, `true`, but in fact, there are multiple errors happening in this code.

In Clojure, sets follow the mathematical definition of a set and thus cannot contain duplicate values. When we add a value that is already in an existing set, we just get the same set in return:

```
(conj #{1 2} 1)
;;=> #{1 2}
```

This is not an error; it simply does not result in a modified set.

However, it is an error to declare a literal set that contains a duplicate value:

```
#{1 2 1}
;; Syntax error reading source at (REPL:13:9).
;; Duplicate key: 1
```

We literally cannot “say” a set that has a duplicate value in it.

But, returning to our teaser, these sets are initialized with actual code, `(rand-int 6)`, and the error we get is the same error: “Duplicate key: `(rand-int 6)`”. This requires a deeper look and some careful thinking about reading, evaluation, and where duplicate checks must occur.

Recall that Clojure's reader takes a string of characters and parses it into Clojure data structures. In this example, the reader reads a set containing two lists where each list contains the symbol `rand-int` and the number 6:

```
#{([rand-int 6]) ([rand-int 6])}
```

Sets are evaluated by recursively evaluating each element of the set.

The important nuance revealed in this teaser is that sets check for duplicate elements at both the read AND evaluation phases. If the duplicate constraint was checked only at read time, then our example could (randomly) return two dice rolls that were the same, and after evaluation, you would have a literal set with duplicates. If the set checked only at evaluation time, then our data structure would need to allow multiple duplicate elements coming out of the read phase, and our data structure does not allow this. Thus, Clojure effectively checks for duplicate set values at both phases.

If you do actually need to execute the same code more than once and collect that code in a set with de-duplication, use `conj` to append to a set rather than creating a literal set:

```
(conj #{} (rand-int 6) (rand-int 6))
;;=> #{4 2}  ;; or something else!
```

This code will never produce an exception, but you may sometimes get a set with only one number (in the case where the two rolls were the same). This highlights that using a set for two dice rolls is not the right collection type to use. A vector would probably be best, and this would also avoid the issues with duplicate checking:

```
[(rand-int 6) (rand-int 6)]
;;=> [3 3]
```

Choosing the right collection type is often the most important choice!

Further Reading

Clojure Reference - Data Structures

https://clojure.org/reference/data_structures

Clojure Reference - Evaluation

<https://clojure.org/reference/evaluation>

Paren-tal Advisory

```
book/src/parental_advisory.clj
```

```
(def names ["Scarlet" "Dandelion" "Cerulean"])
(def codes ["FD0E35" "FED85D" "02A4D3"])
(= (map #([%1 %2]) names codes)
   '(["Scarlet" "FD0E35"] ["Dandelion" "FED85D"] ["Cerulean" "02A4D3"])))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(def names ["Scarlet" "Dandelion" "Cerulean"])
(def codes ["FD0E35" "FED85D" "02A4D3"])
(= (map #(%1 %2) names codes)
   '(["Scarlet" "FD0E35"] ["Dandelion" "FED85D"] ["Cerulean" "02A4D3"]))

;; Execution error (ArityException) at user/eval10$fn (REPL:1).
;; Wrong number of args (0) passed to: clojure.lang.PersistentVector
```

Discussion

The expected answer was pairs of color names and their corresponding hex codes paired into a list of vectors, but this example throws an exception from the inner function instead.

The map function can take any number of collections and walk through them in parallel, sending the next value of each to the provided function. This code takes two collections (names and codes) and uses an *anonymous function* to create a literal vector for each pair of inputs.

In Clojure, anonymous function syntax is indicated with `#( expr )`, and any arguments are conveyed using special parameter symbols. A single parameter (the most common case) will be an unnumbered `%`, while a multi-arity function will use `%N` (with any integer `N`) to refer to the `n`th argument.

In this case, the `[%1 %2]` forms a two-element vector as desired. However, anonymous function syntax uses parentheses around the expression, and parentheses in Clojure almost always represent *invocation*. In Clojure, most data structures are invocable, and vectors, in particular, take a vector index to look up the element value at that index in the vector, like this:

```
([:hi :there] 0) ;; => :hi
```

In the teaser example, the anonymous function constructs the vector `["Scarlet" "FD0E35"]`, but the anonymous function then invokes it without any arguments: `(["Scarlet" "FD0E35"])`. At that point, Clojure throws an `ArityException` to tell the user that the function (the vector) was called with 0 arguments, which is not a valid arity. This was not what we intended.

Instead, you can replace the literal vector syntax with the vector function in the core library, which constructs a vector when invoked:

```
(map #(vector %1 %2) names codes)
;;=> ([Scarlet FD0E35] [Dandelion FED85D] [Cerulean 02A4D3])
```

Or, even better, note that vector itself is already a function that makes vectors, and we don't need an anonymous function at all! This expression is both more concise and easier to read.

```
(map vector names codes)
;;=> ([Scarlet FD0E35] [Dandelion FED85D] [Cerulean 02A4D3])
```

The vector function is invoked with the first name and code and then the second name and code, and so on, to produce a collection of name code pairs for each color.

Further Reading

Clojure Reference - Reader Dispatch Character

https://clojure.org/reference/reader#_dispatch

ClojureDocs - clojure.core/vector

<https://clojuredocs.org/clojure.core/vector>

Clojure Guide - Reading Clojure Characters - Anonymous Functions

https://clojure.org/guides/weird_characters#_anonymous_function

Quoth the Raven

[book/src/quoth.clj](#)

```
(def raven "nevermore")  
  
(= raven "nevermore")  
(= 'raven (symbol "raven"))  
(= `raven (symbol "user/raven"))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output for all expressions:

```
(def raven "nevermore")
(= raven "nevermore")
;;=> true
(= 'raven (symbol "raven"))
;;=> true
(= `raven (symbol "user/raven"))
;;=> true
```

Discussion

All three forms return true, demonstrating several versions of symbol evaluation and quoting.

In Clojure, all code is made up of expressions, which may be arbitrarily nested. The evaluation phase processes each top-level expression to determine a result, which typically involves recursively evaluating nested expressions. However, sometimes we want to read a form but NOT evaluate it, thus leaving it as data. That brings us to *quoting*.

The first form is just the symbol raven:

```
(= raven "nevermore")
;;=> true
```

Symbols are names that resolve to something. Here, the raven symbol is a name referring to the var that was created with def in the first line. The var is resolved in the namespace and evaluated to its current value, the string "nevermore".

The second form quotes the symbol with a single quote using a single quote character:

```
(= 'raven (symbol "raven"))
;;=> true
```

The ' is expanded in the reader to (quote raven). quote is a special form implemented by the compiler—it will cause its value to be read as data by the reader but NOT evaluated.

For example, the following code creates a literal list with symbols without looking up the symbols or invoking the first element of the list as an operation.

```
(a b c)    ;; tries to evaluate by invoking a
;;=> Syntax error compiling at (REPL:1:1).
;;=> Unable to resolve symbol: a in this context

'(a b c)   ;; suppress evaluation
;;=> (a b c)
```

The third form uses a backtick character (called *syntax quote* in Clojure):

```
(= `raven (symbol "user/raven"))
;;=> true
```

Like a quote, a syntax quote avoids evaluation, but it has some additional features that make it ideal for use in macros. Macros are a special kind of function—they take code as input and return replacement code as output. In a sense, macros let users provide code that runs at compilation time (perhaps Poe would call it a “Dream Within a Dream”).

The teaser was run in the user namespace. Because the ``raven` symbol is unqualified and does not refer to a local binding, it is resolved here in the “current” namespace. The syntax quoted symbol is thus resolved to `user/raven`.

Because macros run in the context where they are invoked, they need to return code with names that have meaning in that context, not within the namespace of the macro itself. Thus, macro writers can use a syntax quote with unqualified symbols to resolve to qualified symbols in the context of the expanded code.

In this teaser, we’ve seen how symbols, quoted symbols, and syntax quoted symbols vary in their evaluation. While you won’t need to quote very often in your code, it is an essential tool when writing macros and some kinds of meta-programming.

Further Reading

Clojure Reference - Evaluation

<https://clojure.org/reference/evaluation>

Clojure Reference - Special Form quote

https://clojure.org/reference/special_forms#quote

Loose Threads

[book/src/loose_threads.clj](#)

```
(= (-> 10 inc)  
   (-> 10 #(+ % 1)))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(= (-> 10 inc)
  (-> 10 #(+ % 1)))

;; Syntax error (ClassCastException) compiling fn* at (REPL:2:4).
;; java.lang.Long cannot be cast to clojure.lang.ISeq
```

Discussion

The thread macros `->` and `->>` allow us to structure code as a pipeline of expressions where the result of each is *threaded* into the next. The *thread-first* macro `->` evaluates the first expression and threads its result into the first argument of the next expression and then the result of that into the next expression, and so on.

For example, if you have this nested expression:

```
(* (- (inc 100) 80) 2)
;;=> 42
```

The expressions need to be evaluated inside-out:

```
(inc 100) ;; 101
(- 101 80) ;; 20
(* 20 2) ;; 40
```

The thread-first macro lets us write the same thing but in the order of evaluation instead:

```
(-> 100 inc (- 80) (* 2))
;;=> 42
```

Most developers find it easier to read the left-to-right pipeline with `->` than the inside-out nested expression. This is particularly common when working with collection functions (such as `conj`, `assoc`, and `get`) or single-arg functions.

It is worth noting here one special case in the prior example: a bare symbol (like `inc`) is treated as an expression of one argument which will receive the threaded value, so `inc` is shorthand for: `(inc)`.

`->>` (*thread-last*) is the same as `->` but threads the result into the *last* argument of the next expression instead of the first argument. As it happens, Clojure sequence functions take their input sequence as the last argument, so `->>` is a perfect fit when performing a series of sequence operations:

```
(->> (range 10) (map inc) (filter odd?))
;;=> (1 3 5 7 9)
```

Returning to our teaser, you might expect the thread-first macro to supply the first argument 10 to the next function `#+ % 1`), but it doesn't work. The reason is due to the use of an anonymous function. In Clojure, any expression that evaluates to a function can be put in the "function" position of an expression. That doesn't work here because `->` is a macro, and the anonymous function is not evaluated. While the thread macros look very sophisticated, they are just simple machines that rearrange code from one form to another.

Let's first unroll one bit of syntax—the Clojure reader will rewrite the anonymous function's literal syntax to use `'fn'` instead of `'(fn [x] (+ x 1))'`. So, the expression the thread-first macro sees is `'(-> 10 (fn [x] (+ x 1)))'`. Then, let's remember what `'->'` does—it reorganizes the code so the first expression is the first arg of the next expression. When you do that here, you get: `'(fn 10 [x] (+ x 1))'`.

Suddenly, this looks very wrong—it's an anonymous function named with the number 10 in the place where the function name symbol can appear. And indeed, that explains the error message—Long (10) cannot be cast to ISeq. Here, the macro did not find a symbol, so it tries to resolve the function as a multi-arity function definition. None of this is what you intended; you wanted 10 to be the first argument to the function expression.

There are two common workarounds to this issue: wrapping anonymous expressions in another invocation, and `as->` (*thread-as*).

The first workaround involves wrapping each anonymous function used in another pair of parentheses, turning it into an expression with the same structural similarity as our other expressions:

```
(-> 10 (#(+ % 1)))  
;;=> 11
```

Let's unroll this like we did before to see why it works: `((fn [x] (+ x 1)) 10)`. The new parentheses create a normal Clojure expression with the anonymous function in the function position and the 10 as the first argument to it. Introducing the additional parentheses has the desired effect, but you lose a lot of the clarity that you used the threading macro for in the first place, so this is not ideal.

A better option is to use the *thread-as* macro, `as->`. Use `as->` whenever you want to be explicit about where the previous result should be threaded. This usage works with anonymous functions and with pipelines of expressions where the value is not consistently in the first or last position:

```
(as-> 10 $ (+ $ 1))  
;;=> 11
```

`as->` takes the starting expression, a local binding name, and a series of expressions. The starting expression is evaluated to the local binding name. Then, the expressions are evaluated in order, where the binding name represents the result value of the previous expression. We use `$` as the name here (a popular idiom) because it is short and does not collide with other common syntax, but really, any symbol will work.

The `as->` macro is also helpful when using a mix of first, last, or other position threading:

```
(require '[clojure.string :as str])

(as-> "a:b::c:e:" $
  (str/split $ #":")
  (remove str/blank? $)
  (str/join "-" $))
;=> "a-b-c-e"
```

But, if you find yourself reaching for `as->` for this reason, often there is some better way to write that code with helper functions or more steps for better readability.

Further Reading

Clojure Guide - Threading Macros

https://clojure.org/guides/threading_macros

Part IV

Runtime and Library

Clojure provides a robust set of special forms and functions as part of the standard library. These teasers explore some common places where these operations work differently than you might expect.

A Case of Mistaken Identity

book/src/case_identity.clj

```
(def photo1 :hero)
(def photo2 :villain)

(defn identify
  [photo]
  (case photo
    photo1 "Our hero!"
    photo2 "The dastardly villain"
    "Unknown"))

(= (identify photo1) "Our hero!")
(= (identify photo2) "The dastardly villain")
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(def photo1 :hero)
(def photo2 :villain)

(defn identify
  [photo]
  (case photo
    photo1 "Our hero!"
    photo2 "The dastardly villain"
    "Unknown"))

(= (identify photo1) "Our hero!")
;;=> false

(= (identify photo2) "The dastardly villain")
;;=> false
```

Discussion

We wrote this function at the Clojure Bureau of Investigation, but it's failing to properly identify either the hero or the dastardly villain! The code in this teaser attempts to identify photo1 and photo2 but instead matches neither and returns the fallthrough value "Unknown" in both cases.

Clojure has several options in the core library for choosing conditional behavior—the special form `if`, and macros like `cond`, `condp`, and `case`. `case` is specifically used for determining whether an expression matches one of a set of values in constant time and evaluates the corresponding expression.

Constant time (you may also see it as “O(1)”) means that regardless of the number of expressions, the time to find the match is the same (approximately). Constant time is like looking up a book chapter in the table of contents and going to that page number vs. reading through the book from the beginning until you find it. The `case` function leverages special bytecode support in the JVM that implements this fast mode of dispatch.

The key to the puzzle is that `case` always expects constant values as cases, and these are not evaluated—they are typically things like numbers, strings, keywords, or literal collections of these. Considering symbols, most “normal” Clojure code expects symbols to be evaluated to the value of the symbol in the current context (either a local binding or a namespace var). However, because `case` does not evaluate its case values, symbols just match the literal symbols, not the value from evaluating a symbol.

In our puzzle, the case matches the literal symbols `photo1` and `photo2`, not the values they are bound to:

```
(identify 'photo1)
;;=> "Our hero!"
(identify 'photo2)
;;=> "The dastardly villain"
```

If we want to match the keywords `:hero` and `:villain` with a case, they need to be keywords in the case constants:

```
(defn identify
  [photo]
  (case photo
    :hero "Our hero!"
    :villain "The dastardly villain"
    "Unknown"))
```

Alternately, we can use a conditional form other than case if we need to evaluate vars for the cases:

```
(defn identify
  [photo]
  (cond
    (= photo photo1) "Our hero!"
    (= photo photo2) "The dastardly villain"
    :else "Unknown"))
```

The key thing to remember is that case values are not evaluated—most mistakes with case stem from this.

Further Reading

ClojureDocs - [clojure.core/case](https://clojure.github.io/clojure/clojure.core-api.html#clojure.core/case)

<https://clojure.github.io/clojure/clojure.core-api.html#clojure.core/case>

Boom Goes the Dynamite

```
book/src/boom_goes_the_dynamite.clj
```

```
(defn doubled
  "Returns a sequence of all i's from 0 to n (exclusive), doubled.
  Ex: (doubled 3) ;; (0 0 1 1 2 2)"
  [n]
  (loop [i 0
        output []]
    (if (< i n)
      (recur (inc i)
             (concat output [i i]))
      output)))

(= 0 (first (doubled 50000)))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(defn doubled
  "Returns a sequence of all i's from 0 to n (exclusive), doubled.
  Ex: (doubled 3) ;; (0 0 1 1 2 2)"
  [n]
  (loop [i 0
        output ()]
    (if (< i n)
      (recur (inc i)
             (concat output [i i]))
      output)))

(= 0 (first (doubled 50000)))
;; Execution error (StackOverflowError) at (REPL:1).
```

Discussion

BOOM! What happened?

If we take a careful look at the doubler code, we see a simple loop that's just counting up from 0 to n. At each iteration, we build more of the output sequence by using `concat` with a new pair of elements. It's not obvious, but `concat` is the heart of the problem here.

The `concat` function is implemented with the very important `lazy-seq` macro. Here is a simplified implementation of `concat` that removes sequence chunking and support for multiple collections that aren't relevant:

```
(defn concat1
  ([ ] (lazy-seq nil))
  ([coll] (lazy-seq coll))
  ([coll1 coll2]
   (lazy-seq
    (let [s (seq coll1)]
      (if s
         (cons (first s) (concat1 (rest s) coll2))
         coll2))))))
```

Dropping a little further down the rabbit hole, `lazy-seq` is a macro that takes its body, wraps it into a function, and returns a `seq`. The body invocation is delayed until it's needed.

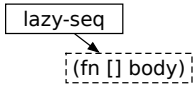
A lazy sequence is progressively evaluated as consumers use it. Once a value is calculated, it is kept and not evaluated again. Initially, a lazy seq holds an unevaluated body (a thunk).

Thunk Alert

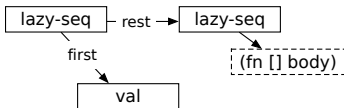


This technique of wrapping an evaluation into a function for later execution is called a *thunk* in functional programming.

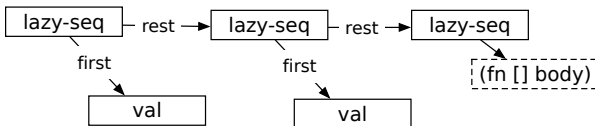
The origin of “thunk” is a joke referring to the past tense of “think,” which doesn’t actually make much sense—it’s really a wrapper around *future* thinking to do.



When you ask for the first element of the lazy sequence, the thunk function holding the body is invoked, and the first element is returned. Typically, the rest of the sequence is another lazy sequence whose own body is another unrealized thunk.



When you need the second element in the lazy sequence, the second thunk function is evaluated. The pattern repeats as the sequence is realized.



In the case of `concat`, the body returns the first of its first collection, combined with another `concat` of the rest of the first collection and the second collection. When the first collection runs out, it just returns the second collection.

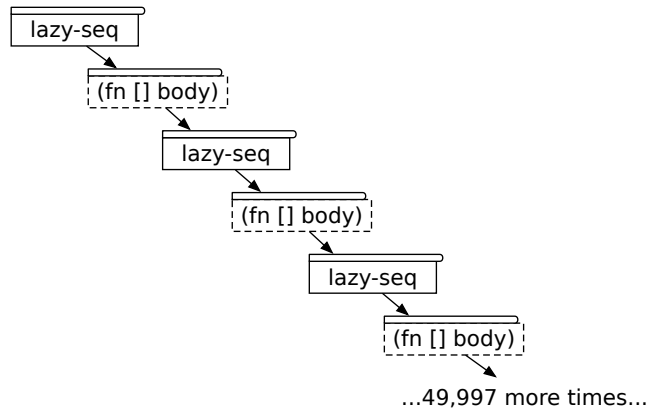
Now, let’s return to the original code, which was doing a `concat` in a loop/recur. The result of that is effectively something like:

```

( ... ;; n times
  (concat
    (concat
      (concat
        (concat ()
          [0 0])
        [1 1])
      [2 2])
    [3 3])
  ...)
```

Every call to `concat` creates a new delayed computation, wrapping the prior one and creating a long chain of pending work. Imagine these delayed compu-

tations like stairs going down to the basement, where we keep all the values, as you can see in the following figure.



When the first element is needed, Clojure has to evaluate the whole staircase of pending computations to get the first value, as each step needs the value from the one below it. The JVM has finite stack memory, so this strategy will eventually fail when n is big enough (the exact number will vary depending on your JVM version and settings). When this happens, we run out of stairs with a `StackOverflowError`.

Clearly, creating a suspended computation 50,000 levels deep is a bad strategy here. Instead, we should do some of the work on each loop iteration rather than saving it all for later.

Eagerly Seq-ing Your Feedback

Rather than creating a lazy sequence, instead, we can eagerly build the final collection as we go, using `loop`.

```

(defn doubled-take2
  [n]
  (loop [i 0
        output []]
    (if (< i n)
      (recur (inc i) (conj output i i))
      output)))

```

Now, the output is a vector (not a sequence), and every step of the loop appends two new values using `conj`. Thus, the result performs a bit of the work each time rather than deferring the computation of all work to access time (and risking a stack overflow later).

This rewrite eagerly computes the collection as it goes rather than lazily saving a recipe on how to compute the final result later. Which is better? As we so frequently learn, “it depends.” Lazy sequences are particularly useful when that laziness allows us to avoid ever doing work that won’t be needed. In this example, what if the consumer of the result of `doubled` only needed to print the first couple of numbers on a user interface? The user might never look at the rest, so why compute them? Similarly, lazy sequences can be infinite (consider a sequence of “all the prime numbers”), and laziness is the essential property that lets us remember the recipe for computing the next value without pre-computing all of them.

However, it is perhaps more common that we DO need all of the collection computed, and in that case, we may be able to compute that more efficiently or more safely, as in `doubled-take2` (avoiding a pending computation that’s too big).

Further Reading

“Clojure Don’ts: Concat” by Alessandra Sierra

<https://stuartsierra.com/2015/04/26/clojure-donts-concat>

Banana Splits

```
book/src/banana_splits.clj
(require '[clojure.string :as str])

(= (str/split "banana" #"an") ["b" "" "a"])
(= (str/split "banana" #"na") ["ba"])
```

Guess the Output



Try to guess what the output is before moving to the next page.

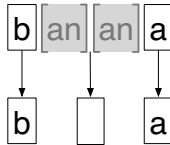
The program will display the following output:

```
(require '[clojure.string :as str])
(= (str/split "banana" #"an") ["b" "" "a"])
;;=> true
(= (str/split "banana" #"na") ["ba"])
;;=> true
```

Discussion

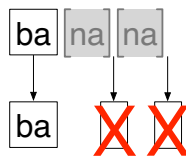
Everyone loves a banana split, but how do you split it? In this teaser, we split the string "banana" into parts using different regular expression matches as the dividers.

Considering the first regular expression `"#an"`, the `"banana"` matches two `"an"` expressions and yields three string parts, including an intermediate empty string.



Thus, splitting with `"#an"` yields `["b" "" "a"]`.

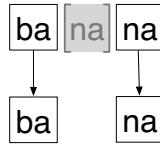
The second regular expression may have produced a surprising result, with only a single part.



You might have expected `["ba" "" ""]`, but the default behavior of `split` omits trailing empty string parts.

The limit Parameter

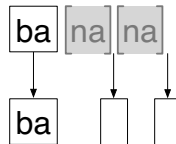
The `split` function also takes an optional `limit` parameter that controls the number of times the split pattern should be applied. When the limit `N` is positive, the pattern will be applied, at most, `N-1` times. For example, to apply the pattern no more than once, call `(str/split "banana" #"na" 2)`.



This will split on only the first "na", returning ["ba" "na"].

When the limit is 0, the pattern matches any number of times, and trailing empty strings are omitted (the default behavior).

When the limit is negative, the pattern matches any number of times and trailing empty string parts are retained. Returning to the second example in the original teaser, passing -1 can thus keep the empty trailing parts, returning ["ba" "" ""].



Most of this functionality derives from the `java.lang.String` `split()` method in the JDK.

split-lines

Another related function in the `clojure.string` namespace is `split-lines`, which will take a string and split it into lines using a regular expression for newline characters as follows.

```
(str/split-lines "a\nb\nc\n\n")
;;=> ["a" "b" "c"]
```

Because `split-lines` is built on `split`, it has the same behavior in dropping trailing empty lines. Thus, combining `split-lines` and `join` with a newline character will not round-trip back to the original string. This is a common confusion for developers using `split-lines` to split apart a string, modify it, and reconstitute a replacement string:

```
(->> "a\nb\nc\n\n"
  str/split-lines
  (map str/upper-case)
  (str/join "\n"))
;;=> "A\nB\nC"
```

This is a common pattern that developers use to process multi-line text:

- Call `split-lines` to split text into lines
- Modify each line via `map`
- Call `str/join` to recombine the lines

It is often surprising that `split-lines` removes trailing newlines. The `split-lines` function does not currently take a limit parameter, but you can use `split` directly with a limit to get the same effect of retaining empty lines at the end:

```
(defn split-lines-trailing [s]
  (str/split s #"\\n" -1))

(->> "a\\nb\\nc\\n\\n"
  split-lines-trailing
  (map str/upper-case)
  (str/join "\\n"))
;;=> "A\\nB\\nC\\n\\n"
```

With this modification, `split-lines-trailing` retains the empty lines at the end.

Further Reading

ClojureDocs - clojure.string/split

<https://clojuredocs.org/clojure.string/split>

Java Platform API - [java.lang.String/split](https://docs.oracle.com/javase/8/docs/api/java/lang/String.html#split-java.lang.String-int-)

<https://docs.oracle.com/javase/8/docs/api/java/lang/String.html#split-java.lang.String-int->

Hanging Around

book/src/hanging_around.clj

```
(require '[clojure.java.io :as jio]
         '[clojure.string :as str])

(defn grep
  "Return lazy sequence of matches to substring
  in the file at path"
  [substring path]
  (with-open [reader (jio/reader path)]
    (->> reader
      (line-seq)
      (filter #(str/includes? % substring)))))

(grep "defn" "book/src/hanging_around.clj")
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(require '[clojure.java.io :as jio]
         '[clojure.string :as str])

(defn grep
  "Return lazy sequence of matches to substring
  in the file at path"
  [substring path]
  (with-open [reader (jio/reader path)]
    (->> reader
      (line-seq)
      (filter #(str/includes? % substring)))))

(grep "defn" "book/src/hanging_around.clj")

;; Error printing return value (IOException) at
;;   java.io.BufferedReader/ensureOpen (BufferedReader.java:122).
;; Stream closed
```

Discussion

The program presented here is similar to the Unix program `grep`, which is used to find lines matching a substring in a file. The expected behavior is to return a lazy sequence of all lines matching the substring in the file at the path, but instead, we receive an exception.

The function does indeed return a lazy sequence, but that is actually the problem here. The `with-open` macro is a helper that opens a closeable resource, executes the body, and then closes the resource. The lazy sequence returned by the body has not yet been executed, and the file has not been read when `with-open` closes the file stream.

When the REPL printer goes to print the lazy sequence, it finally tries to read the resource and discovers the stream has already been closed. In general, lazy sequences do not mix well with external I/O resources because there is no good way to convey that the resource should be closed, which means either closing it too early (as we did here), not closing it at all (leaving it hanging), or needing to explicitly close the resource later after all of the data has been processed.

Eager Processing

One solution is to eagerly process all of the data, close the resource, and then return. The easiest modification to the program is to wrap the sequence expression in `doall` to force full construction of the result before the resource is closed.

```
(defn grep-eager
  "Return realized sequence of matches to substring in
  the file at path"
  [substring path]
  (with-open [reader (jio/reader path)]
    (doall
      (->> reader
        (line-seq)
        (filter #(str/includes? % substring))))))

(grep-eager "defn" "book/src/hanging_around.clj")
;;=> ("(defn grep" "(grep \"defn\" \"book/src/hanging_around.clj\")")
```

This version matches the two lines containing "defn" and returns them, but it does all the work before returning any answer. How can we retain the laziness?

Caller Closes Reader

The problem here is that 'grep' closes the reader and returns a sequence that still (lazily) plans to use it. One option is to relieve 'grep' of the responsibility of closing and give that back to the caller, which knows better when it's done using it.

```
(defn grep-reader
  "Return lazy sequence of matches to substring
  in the reader.
  Caller is responsible for closing reader."
  [substring reader]
  (->> reader
    (line-seq)
    (filter #(str/includes? % substring))))

(with-open [rdr (jio/reader "book/src/hanging_around.clj")]
  (doall (grep-reader "defn" rdr)))
;;=> ("(defn grep" "(grep \"defn\" \"book/src/hanging_around.clj\")")
```

This version lets the grep-reader function focus on one job—grepping. The caller is responsible for creating and closing the reader (here with with-open) and is responsible for reading all of the results with doall (but it could just as easily read less). Often, it's a good idea to pair the construction of stateful resources with the code that closes them. In all cases, it's good to be clear in your docstrings who owns the responsibility.

Further Reading

ClojureDocs - clojure.core/with-open
<https://clojuredocs.org/clojure.core/with-open>

ClojureDocs - clojure.core/doall
<https://clojuredocs.org/clojure.core/doall>

Build a Nest

`book/src/build_nest.clj`

```
(def ex1
  (for [a [0 1 2]
        b [3 4 5]]
    (* a b)))

(def ex2
  (for [a [0 1 2]]
    (for [b [3 4 5]]
      (* a b))))

(= ex1 ex2)
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(def ex1
  (for [a [0 1 2]
        b [3 4 5]]
    (* a b)))

(def ex2
  (for [a [0 1 2]]
    (for [b [3 4 5]]
      (* a b))))

(= ex1 ex2)
;;=> false
```

Discussion

As you might have guessed, these examples are similar but, importantly, different in what they produce. Let's compare `ex1` (a `for` with two binding forms) and `ex2` (a nested `for`):

```
ex1
;;=> (0 0 0 3 4 5 6 8 10)

ex2
;;=> ((0 0 0) (3 4 5) (6 8 10))
```

We see the same numbers but different collection nesting! Let's dive into the details of `for` to gain intuition for why these are what they are.

If you are more familiar with languages like Java or JavaScript, those languages use `for` as a looping construct where the body is executed once per iteration until a condition is met.

In Clojure, `for` is a sequence generator (you can impress programming language nerds by calling them *list comprehensions*). A `for` expression in Clojure always yields one lazy, possibly infinite, sequence.

Given a `for` like:

```
(for [x collection]
  expr)
```

Read this as “for each binding of `x` to a value of `collection`, generate a sequence of `expr` evaluations.” These evaluations are done lazily to generate the next value in the lazy sequence that is returned.

When there are multiple bindings like:

```
(for [x collection1
      y collection2]
  expr)
```

Read this as “for each binding of *x* to a value in *collection1*, then for each binding of *y* to a value in *collection2*, generate a sequence of *expr* evaluations.” Given our *ex1* definition, this creates the following bindings and expression results:

```
a = 0  b = 3  expr = 0
a = 0  b = 4  expr = 0
a = 0  b = 5  expr = 0

a = 1  b = 3  expr = 3
a = 1  b = 4  expr = 4
a = 1  b = 5  expr = 5

a = 2  b = 3  expr = 6
a = 2  b = 4  expr = 8
a = 2  b = 5  expr = 10
```

But remember, *every* for generates a single sequence, so these results will all be returned in the same sequence, even though the evaluation is nested:

```
(0 0 0 3 4 5 6 8 10)
```

So, how does *ex2* differ? In this case, we nest two *for* expressions instead of one for with two bindings. The nested expression evaluation will be exactly the same as *ex1*, but the difference lies in having two *for* expressions. Each *for* is a sequence generator, so the inner *for* will generate a new sequence for every value of the outer *a* value:

```
((0 0 0) (3 4 5) (6 8 10))
```

The *for* expression has a lot of additional tricks you can use to include only certain binding sets (*:when*), create intermediate results (*:let*), or terminate early (*:while*). We won’t cover those here, but you should experiment in your REPL to see all the things it can do.

Further Reading

ClojureDocs - [clojure.core/for](https://clojure.org/core/for)

<https://clojuredocs.org/clojure.core/for>

Partial Recall

```
book/src/partial_recall.clj
(defn sentence [subject object]
  (str "Is " subject " a " object "?"))

(def planet "Mars")

(def f1 (partial sentence planet))
(def f2 (fn [object] (sentence planet object)))

;; redefine
(def planet "Earth")

(= (f1 "dream") (f2 "dream"))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(defn sentence [subject object]
  (str "Is " subject " a " object "?"))

(def planet "Mars")

(def f1 (partial sentence planet))
(def f2 (fn [object] (sentence planet object)))

;; redefine
(def planet "Earth")

(= (f1 "dream") (f2 "dream"))
;;=> false
```

Discussion

The output from f1 (based on partial) and f2 (based on an anonymous function) are different:

```
(f1 "dream") ;; "Is Mars a dream?"
(f2 "dream") ;; "Is Earth a dream?"
```

The partial function takes a function of N arguments and some initial arguments and returns a new function that calls the function with the initial arguments and any remaining arguments supplied at evaluation time:

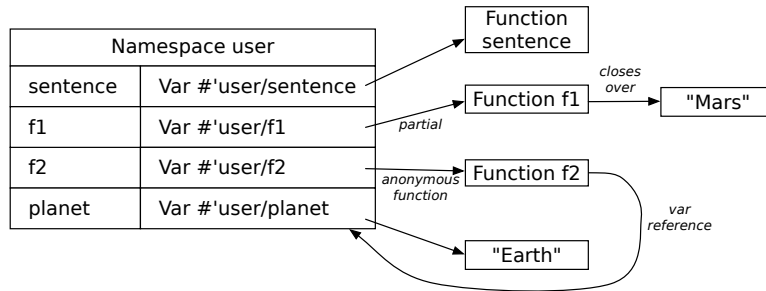
```
(def double (partial * 2))
(double 10) ;; 20
```

The double function is a partial application of *, providing the initial argument 2. So (double 10) evaluates (* 2 10).

In Clojure, all function parameters are evaluated before the function is invoked. So, if we look back to the teaser, f1 evaluates planet when f1 is defined. Clojure evaluates a symbol by looking it up in the current namespace and returning the value of the var at that symbol.

A *var* is a “box”—a place that holds a value. You create vars with def (a special form) or other expressions that ultimately call def (most of these start with def to make that clear—defn, defmacro, defrecord, defmulti, and so on).

This diagram shows the namespace mapping at the end of the teaser. The table shows a mapping of name to var in the current namespace. You can see a representation of this table at your REPL by calling (ns-interns *ns*).



Our example calls `def` several times. The definition of `f1` evaluates (partial sentence planet). Clojure evaluates `planet` by looking up that name in the current namespace and getting the planet string value. The definition of `f2` defines a new anonymous function. Clojure creates a new class for the function with a reference to the `#'planet` var. This var is looked up only when `f2` is invoked. Thus, the `f2` function will “see” changes to `planet` after its initial definition.

The final call to `def` does not create a new var for `planet`; instead, it puts a new value in the existing var. This is where the behavior diverges. When `f1` is called, it uses the function object it found at definition time. When `f2` is invoked, it runs the anonymous function, which includes a reference to the `planet` var, which looks up the var and finds the modified value.

This difference can sometimes be confusing when developing at the REPL and using `partial`. If you are redefining the function given to `partial`, you won’t see those changes when the original value has been captured by `partial`.

Some other function-making functions that can have the same effects include `comp`, `complement`, and `juxt`.

Further Reading

Clojure Reference - Vars

<https://clojure.org/reference/vars>

Abrac-OR-dabra

[book/src/abracordabra.clj](#)

```
(def flip {:head :tail, :tail :head})

(let [empty {}
      left nil, right nil
      {:keys [left right] :or {left :head, right (flip left)}} empty
      right-1 right
      left nil, right nil
      {:keys [right left] :or {left :head, right (flip left)}} empty
      right-2 right]

  (println "Results:" right-1 right-2)
  (= right-1 right-2))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(def flip {:head :tail, :tail :head})

(let [empty {}
      left nil, right nil
      {:keys [left right] :or {left :head, right (flip left)}} empty
      right-1 right
      left nil, right nil
      {:keys [right left] :or {left :head, right (flip left)}} empty
      right-2 right]

  (println "Results:" right-1 right-2)
  (= right-1 right-2))

;; Results: :tail nil
;;=> false
```

Discussion

In this magic trick, one small change manifests surprisingly different results. To set the coins in left and right “hands” the wizard destructures an empty map.

The magic of destructuring improves code readability and reduces mental effort by creating multiple local bindings with simple declarative syntax. When used appropriately, destructuring *can* keep your code clean and concise. In this showy spellcasting, *associative destructuring* defines local bindings for both left and right hands in a single step.

The `:keys [left right]` portion of the destructuring is equivalent to:

```
(let [...
      left (get empty :left)
      right (get empty :right)]
  ... )
```

This is the only part of the magic trick that changes in the two examples, binding either left then right or right then left. Typically, this has no effect on the result.

The `:or` clause in associative destructuring is used to provide a default value if the specified symbol is not found in the data structure. In the puzzle, the first example destructuring with the `:or` expands to:

```
(let [...
      left (get empty :left :head)
      right (get empty :right (flip left))]
  ... )
```

The second one is similar but swaps the order of the bindings:

```
(let [...
      right (get empty :right (flip left))
      left (get empty :left :head)]
  ... )
```

The difference in the right value is thus dependent on whether left has been bound from empty yet.

This puzzle relies on implementation details of destructuring in Clojure, but there's no guarantee when the `:or` expressions are evaluated. Thus, using this kind of `:or` code is not recommended. Specifically, `:or` clause expressions may depend on local bindings in scope but should not refer to locals being defined by the destructuring itself.

Instead, it's better to make data manipulation explicit in the `let`, and guarantee when the `flip` is evaluated:

```
(let [empty {}
      left nil
      right nil
      {:keys [left right] :or {left :head}} empty
      right (or right (flip left))]
  right)
```

In this version, `right` is destructured, and then an explicit step chooses whether to flip or not. Because no `:or` expressions depend on the values being bound, the order of the `:keys` doesn't matter.

Further Reading

Clojure Reference - Binding Forms (Destructuring)

<https://clojure.org/reference/special-forms#binding-forms>

Clojure Guide - Destructuring

<https://clojure.org/guides/destructuring>

Take the Hint

```
book/src/take_the_hint.clj  
(def ^double PI 3.14159)  
(= 'double (-> #'PI meta :tag))
```

Guess the Output



Try to guess what the output is before moving to the next page.

The program will display the following output:

```
(def ^double PI 3.14159)
(= 'double (-> #'PI meta :tag))
;;=> false
```

Discussion

This code attempts to tag the PI var as primitive type double, but it will actually get tagged as something completely different:

```
(-> #'PI meta :tag)
;;=> #object[clojure.core$double 0x6b7906b3 "clojure.core$double@6b7906b3"]
```

What is that? When the Clojure printer encounters a Java object without a print method, it prints the class name, identity hashCode, and toString(). The returned value being printed is the double function object implementing clojure.core/double, the same object you get from evaluating the symbol double:

```
double
;;=> #object[clojure.core$double 0x6b7906b3 "clojure.core$double@6b7906b3"]
```

Because type hints are just hints and are not required to match the value, nothing prevents you from using the PI var like this; it just does not have the intended primitive type hint.

You may be more familiar with providing type hints on function arg lists or in Java interop calls—in those positions, metadata is not evaluated. The Clojure compiler will see primitive type hints double or long as special type hints corresponding to the primitive classes for doubles and longs:

```
(defn double-doubler
  ^double [^double x]
  (* 2 x))
```

The previous code creates a function that takes and returns a primitive double.

Type hints on vars

With def, metadata is supplied on the var symbol:

```
(def ^double PI 3.14159)
```

The difference here is that metadata applied to the var symbol is evaluated. Because the metadata is evaluated, the special type hint double is evaluated

to the double function, which is not a valid type hint. Type hints that are invalid are simply ignored, so this type of hint has no effect.

The proper way to provide this type is to quote it so that it is not evaluated. The `^` syntax does not expect to read a quote form so you must use the full metadata map syntax:

```
(def ^{:tag 'double} PI 3.14159)
;;=> #'user/PI
(-> #'PI meta :tag)
;;=> double
```

Note that obtaining the metadata `:tag` now returns the special double type hint, not the double function object.

This var is now useful in providing overload selection when used in combination with Java interop calls:

```
(set! *warn-on-reflection* true)
(Math/abs PI)
;;=> 3.14159    ;; no reflection warning
```

However, var values are always Java objects, and the var value is thus a boxed double, not a primitive double, and this call to `Math/abs` will require unboxing. The Java bytecode will look something like this:

```
;; Look up and retrieve the var root value (a Double)
getstatic      user$fn_line_1__254.const__0:Lclojure/lang/Var;
invokevirtual  clojure/lang/Var.getRawRoot:()Ljava/lang/Object;
;; Cast value to Number
checkcast      Ljava/lang/Number;
;; Coerce Number to double (unbox)
invokestatic   clojure/lang/RT.doubleCast:(Ljava/lang/Object;)D
;; Invoke Math.abs()
invokestatic   java/lang/Math.abs:(D)D
```

Can we do better?

:const vars

Clojure provides an additional metadata hint for vars that are constant values. By adding the metadata `:const` flag, you instruct the compiler that this var value is a constant and can be inlined at the point of use:

```
(def ^{:tag 'double, :const true} PI 3.14159)
(Math/abs PI)
;;=> 3.14159    ;; no reflection, primitive value inlined
```

The underlying bytecode will now look like:

```
;; Const value inlined, no var lookup, no unboxing:  
ldc2_w      3.14  
;; Invoke Math.abs()  
invokestatic java/lang/Math.abs:(D)D
```

For true constant values, this is the best option. However, `:const` is a feature that should be used sparingly. Because this inlining happens at compile-time, applications that use `:const` with expressions that derive values from configuration or runtime state will “bake in” values computed too early.

For example, a var that holds a logging level pulled from config files would be computed at compile-time, and the resulting compiled code would never read the logging level at runtime (as expected). In most cases, the use of `:const` should be confined to literal values like numbers or strings, not computed values.

Further Reading

Clojure Reference - Vars

<https://clojure.org/reference/vars>

Clojure Reference - Java Type Hints

https://clojure.org/reference/java_interop#typehints

Bibliography

- [HB18] Alex Miller with Stuart Halloway and Aaron Bedra. *Programming Clojure, Third Edition*. The Pragmatic Bookshelf, Dallas, TX, 2018.
- [Ols18] Russ Olsen. *Getting Clojure*. The Pragmatic Bookshelf, Dallas, TX, 2018.
- [VM15] Ben Vandgrift and Alex Miller. *Clojure Applied*. The Pragmatic Bookshelf, Dallas, TX, 2015.

Index

SYMBOLS

#() anonymous functions, [62](#)
' (single quotes), quoting with, [66](#)
-> thread-first macro, [70](#)
->> thread-last macro, [70](#)
= equality operator, [3-5](#)
== equality operator, [3-5](#)
#_discard reader, [51-53](#), [56](#)
` (backtick), syntax quoting with, [67](#)

A

and macro, vacuous truth, [19-21](#)
anonymous functions, [61-63](#), [71](#)
arbitrary precision numbers, integer overflow, [7-10](#)
arithmetic overflow, [7-10](#)
arrays, Java, [26](#)
as-> thread-as macro, [71](#)
assoc, maps, [42](#)
associative destructuring, [101-103](#)

B

backtick (`), syntax quoting with, [67](#)
Bagwell, Phil, [30](#)
BigInt values, [8](#)
bindings
 destructuring, [101-103](#)
 discard reader (#_), [52](#)
 for expression, [93-95](#)

Booleans

 comparator functions, [46](#)
 logical truth, [16](#)

bounds checking, overflow, [8](#)

C

case, [75-77](#)

Clojure

 as hosted language, [ix](#)
 resources on, [xi](#)
 running REPL, [x](#)

code, for this book, [x](#)

collections

 conj, [29-32](#)
 constructing from nil, [33-35](#)
 equality, [25-27](#)
 Java, [26](#), [35](#)
 searching sequential, [37-40](#)
 sorted, [45-49](#)
 transducers, [31](#)

comment blocks

 nil return, [55-56](#)
 rich, [56](#)

comp, [99](#)

comparators

 compare function as default, [47](#)
 custom, [48](#)
 sorted collections, [45-49](#)

compare, [47](#)

complement, [99](#)

concat, [79-83](#)

conj, [29-32](#), [59](#)

:const flag, [107](#)

constant time, [76](#)

contains?, [37-40](#)

D

destructuring, [101-103](#)

discard reader (#_), [51-53](#), [56](#)

dissoc, [42](#)

doall, [90](#)

docstrings, [38](#)

doubles, NaN, [12](#)

E

empty, [42](#)

equality

 collections, [25-27](#)
 equality operators, [3-5](#), [11-13](#)
 NaN, [11-13](#)

every?, vacuous truth, [19-21](#)

F

false

 Boolean values, [16](#)
 collection comparisons, [26](#)
 logical, [15-17](#)
 or macro, [20](#)

filter, [31](#), [39](#)

filterv, [31](#)

floating-point numbers, NaN, [12](#)

for expression, [93-95](#)

functions

 anonymous, [61-63](#), [71](#)
 comparators, [45-49](#)
 docstrings, [38](#)

partial functions, 97–99
polymorphic, 34

G

Getting Clojure, xi

H

hash trees, 30
hosted languages, Clojure as, ix

I

integers, overflow, 7–10
into, collections, 31
invocation and parentheses
(()), 62

J

Java
 arrays, 26
 collections, 26, 35
 comparator functions, 46
 interop and nil, 35
 interop and type hints, 107
juxt, 99

L

lazy sequences, 79–83, 89–94
lazy-seq macro, 80
let
 destructuring with, 103
 discard reader (#_), 52
:let in for expressions, 95
lines, splitting strings, 87
linked lists, 30
list comprehensions, 94
lists
 conj, 30
 linked, 30
logical true/false, 15–17
loop, 82

M

macros
 syntax quoting, 67
 thread macros, 69–72
map, 31
maps
 collection comparisons, 26
 emptying, 42
 NaN, 12
 records, 41–43
mapv, 31

N

NaN, equality operators, 11–13
nil
 checking for, 16
 comment blocks, 55–56
 constructing collections
 from, 33–35
 or macro, 20
numbers
 arbitrary precision, 7–10
 equality operators, 3–5
 integer overflow, 7–10

O

or, vacuous truth, 19–21
:or clause, associative destructuring, 102
order
 collection comparisons, 26
 compare function, 47
 sets, 32
overflow, arithmetic, 7–10

P

partial functions, 97–99
partitions, collection comparisons, 26
polymorphic functions, 34
Programming Clojure, xi
promotion, arithmetic overflow, 8

Q

quoting, 65–67

R

reader
 #() anonymous functions, 62
 discard (#_), 51–53, 56
 symbolic values and ##NaN, 12
 syntax quoting with ` (backtick), 67
records, adding/removing fields, 41–43
regular expressions, splitting strings, 85–88
REPL, running, x
resources
 closing, 89–92
 lazy sequences, 90

S

search
 linear, 39
 sequential collections
 with contains?, 37–40
sequence generators, 94
sequences
 chunked, 39
 conj, 30
 lazy, 79–83, 89–94
sequential collections
 comparisons, 26
 contains?, 37–40
sets
 collection comparisons, 26
 conj, 32, 59
 contains?, 38
 duplication in, 57–59
 NaN, 12
 order, 32
single quotes (), quoting with, 66
some, 39
sorted collections, comparators, 45–49
sorted-map, 46
sorted-set, 46
speed
 inserting elements with conj, 30
 promoting arithmetic operations, 9
 searching sets with contains?, 39
split, 85–88
split-lines, 87
strings, splitting, 85–88
symbolic values, ##NaN, 12
symbols
 case, 76
 quoting, 65–67
syntax quoting, 67

T

thread macros, 69–72
thread-as macro (as->), 71
thread-first macro (->), 70
thread-last macro (->>), 70
thunks, 81
time, constant, 76
transducers, collections, 31
trees, vectors and conj, 30
tries, 30

truth

- Boolean values, 16
- logical, 15–17, 20
- or macro, 20
- vacuous, 19–21

type hints, 105–108

U

unchecked operations, 8–9

V

vacuous truth, 19–21

vars

- partial functions, 98
- type hints, 106–108

vector function, 62

vectors

- anonymous functions,
61–63

conj, 30

creating, 62

W

:when in for expressions, 95

:while in for expressions, 95

with-open macro, 90

Thank you!

We hope you enjoyed this book and that you're already thinking about what you want to learn next. To help make that decision easier, we're offering you this gift.

Head on over to <https://pragprog.com> right now, and use the coupon code BUYANOTHER2025 to save 30% on your next ebook. Offer is void where prohibited or restricted. This offer does not apply to any edition of *The Pragmatic Programmer* ebook.

And if you'd like to share your own expertise with the world, why not propose a writing idea to us? After all, many of our best authors started off as our readers, just like you. With up to a 50% royalty, world-class editorial services, and a name you trust, there's nothing to lose. Visit <https://pragprog.com/become-an-author/> today to learn more and to get started.

Thank you for your continued support. We hope to hear from you again soon!

The Pragmatic Bookshelf



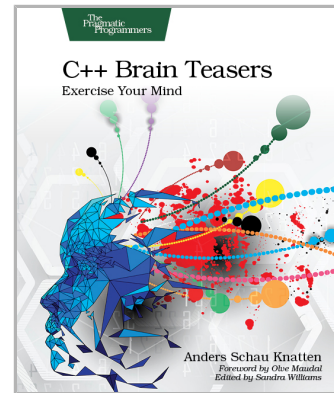
C++ Brain Teasers

C++ is famous for getting all the default behaviors wrong and for sometimes making demons fly out of your nose. Through 25 puzzles, from the useful to the outright weird, we explore some of C++'s most interesting quirks. How does initialization actually work? Do temporaries even exist? Why is `<ins>!!""` a valid expression in C++? As you work through each puzzle, you will peel off some of the layers of complexity of C++, getting a fundamental understanding of how the language works. This will help you write better code and recognize issues more easily while debugging.

Anders Schau Knatten

(138 pages) ISBN: 9798888650516. \$32.95

<https://pragprog.com/book/akbrain>



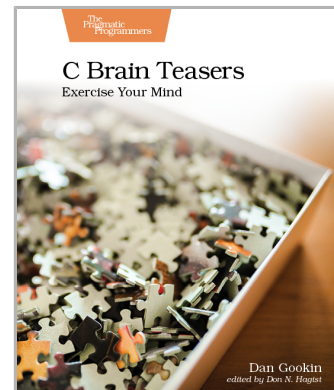
C Brain Teasers

You thought you knew C, but can you solve 25 puzzles in this popular programming language? Noted C programmer and author Dan Gookin provides a series of pointed questions, puzzles, and problems to keep your C programming skills sharp. Each one will provide insight into various aspects of handling strings, numeric operations, and other activities, giving you techniques to take the best advantage of all C has to offer. Challenge yourself, and get to know some powerful tricks and details for writing better, faster, more accurate C code.

Dan Gookin

(118 pages) ISBN: 9798888650486. \$32.95

<https://pragprog.com/book/cbrain>



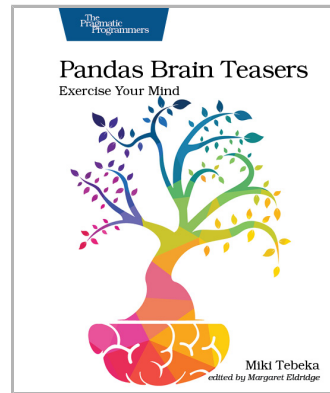
Pandas Brain Teasers

This book contains 25 short programs that will challenge your understanding of Pandas. Like any big project, the Pandas developers had to make some design decisions that at times seem surprising. This book uses those quirks as a teaching opportunity. By understanding the gaps in your knowledge, you'll become better at what you do. Some of the teasers are from the author's experience shipping bugs to production, and some from others doing the same. Teasers and puzzles are fun, and learning how to solve them can teach you to avoid programming mistakes and maybe even impress your colleagues and future employers.

Miki Tebeka

(110 pages) ISBN: 9781680509014. \$18.95

<https://pragprog.com/book/d-pandas>



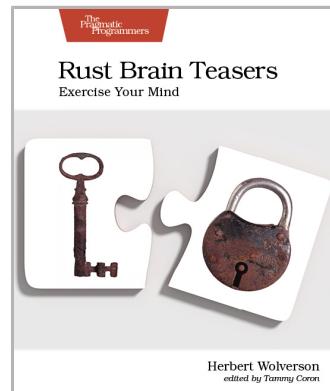
Rust Brain Teasers

The Rust programming language is consistent and does its best to avoid surprising the programmer. Like all languages, though, Rust still has its quirks. But these quirks present a teaching opportunity. In this book, you'll work through a series of brain teasers that will challenge your understanding of Rust. By understanding the gaps in your knowledge, you can become better at what you do and avoid mistakes. Many of the teasers in this book come from the author's own experience creating software. Others derive from commonly asked questions in the Rust community. Regardless of their origin, these brain teasers are fun, and let's face it: who doesn't love a good puzzle, right?

Herbert Wolverson

(138 pages) ISBN: 9781680509175. \$18.95

<https://pragprog.com/book/hwrustbrain>



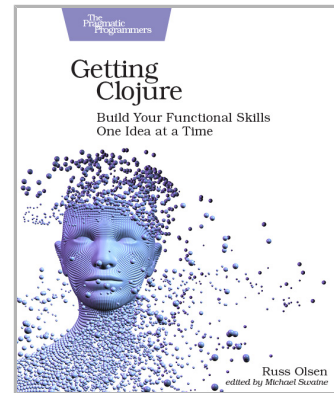
Getting Clojure

Behind every programming language lies a *vision* of how programs should be built. The vision behind Clojure is of a radically simple language framework holding together a sophisticated collection of programming features. Learning Clojure involves much more than just learning the mechanics of the language. To really *get* Clojure you need to understand the *ideas* underlying this structure of framework and features. You need this book: an accessible introduction to Clojure that focuses on the *ideas* behind the language as well as the practical details of writing code.

Russ Olsen

(288 pages) ISBN: 9781680503005. \$47.95

<https://pragprog.com/book/roclojure>



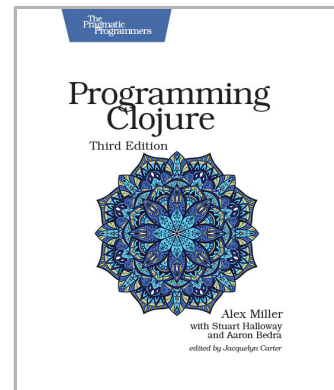
Programming Clojure, Third Edition

Drowning in unnecessary complexity, unmanaged state, and tangles of spaghetti code? In the best tradition of Lisp, Clojure gets out of your way so you can focus on expressing simple solutions to hard problems. Clojure cuts through complexity by providing a set of composable tools—immutable data, functions, macros, and the interactive REPL. Written by members of the Clojure core team, this book is the essential, definitive guide to Clojure. This new edition includes information on all the newest features of Clojure, such as transducers and specs.

Alex Miller with Stuart Halloway and Aaron Bedra

(302 pages) ISBN: 9781680502466. \$49.95

<https://pragprog.com/book/shcloj3>



The Pragmatic Bookshelf

The Pragmatic Bookshelf features books written by professional developers for professional developers. The titles continue the well-known Pragmatic Programmer style and continue to garner awards and rave reviews. As development gets more and more difficult, the Pragmatic Programmers will be there with more titles and products to help you stay on top of your game.

Visit Us Online

This Book's Home Page

<https://pragprog.com/book/mmclobrain>

Source code from this book, errata, and other resources. Come give us feedback, too!

Keep Up-to-Date

<https://pragprog.com>

Join our announcement mailing list (low volume) or follow us on Twitter @pragprog for new titles, sales, coupons, hot tips, and more.

New and Noteworthy

<https://pragprog.com/news>

Check out the latest Pragmatic developments, new titles, and other offerings.

Buy the Book

If you liked this ebook, perhaps you'd like to have a paper copy of the book. Paperbacks are available from your local independent bookstore and wherever fine books are sold.

Contact Us

Online Orders: <https://pragprog.com/catalog>

Customer Service: support@pragprog.com

International Rights: translations@pragprog.com

Academic Use: academic@pragprog.com

Write for Us: <http://write-for-us.pragprog.com>